Teoría de Algoritmos
(TB024) - Curso Echevarría

# Trabajo Práctico 1

Fecha de entrega:
26 de abril de 2026

Integrantes:


Brizuela Sebastian
105288
Chen Nicolas
105907
Janon Santiago Ignacio
106079
Lanseros Nicolas
111762
Ordoñez Joaquin
108865

# ÍNDICE

**Problema 1 - División y conquista:** ..... **5**
1. Introducción: ..... 5
2. Definición del problema: ..... 5
3. Análisis del problema: ..... 5
3.1. El enfoque clásico y su caso de falla: ..... 5
3.2. La estrategia óptima: División en tercios: ..... 6
4. Algoritmo: ..... 6
4.1. Formulación matemática: ..... 6
4.2. Demostración de optimalidad: ..... 7
4.3. Pseudocódigo: ..... 8
4.4. Implementación: ..... 9
4.5. Complejidad: ..... 10
4.5.1. Complejidad Temporal: ..... 10
4.5.2. Análisis de la Cantidad de Pesadas: ..... 11
4.5.3. Complejidad Espacial(auxiliar): ..... 11
4.6. Relación entre Tiempo de Ejecución y Cantidad de Pesadas ..... 11
5. Ejemplo de ejecución y seguimiento: ..... 12
Nivel de recursión 0 (Inicial) ..... 12
Nivel de recursión 1 ..... 12
Nivel de recursión 2 (Caso Base) ..... 13
6. Mediciones: ..... 13
6.1. Validaciones empíricas ..... 14
7. Conclusión: ..... 16
**Problema 2 - Greedy:** ..... **17**
1. Introducción: ..... 17
2. Definición del problema: ..... 17
3. Análisis del problema: ..... 18
3.1 Problema que resuelve y algoritmo replicado ..... 18
3.2 Estructuras de datos utilizadas: ..... 18
4. Algoritmo: ..... 19
4.1. Justificación de la naturaleza Greedy ..... 19
4.2. Regla de selección Greedy ..... 19
4.3. Formulación matemática: ..... 19
4.2. Demostración de optimalidad: ..... 20
4.3. Pseudocódigo: ..... 20
4.4. Implementación: ..... 21
4.5. Complejidad: ..... 23
4.1. Complejidad Temporal: ..... 23
4.2. Complejidad Espacial: ..... 23
5. Ejemplo de ejecución y seguimiento: ..... 24

6. Mediciones y validaciones empíricas:....28
7. Conclusión:....29
**Problema 3 - Backtracking:....30**
1. Introducción:....30
2. Definición del problema:....30
3. Análisis del problema:....30
3.1 Primer enfoque de la solución....30
3.2 El enfoque por Backtracking....31
4. Algoritmo:....31
4.1. Formulación matemática:....32
4.2. Demostración de correctitud y completitud:....32
4.3. Estructuras de datos y control de estado:....33
4.4. Arquitectura de funciones y lógica general:....33
4.5. Pseudocódigo:....34
4.6. Implementación:....35
4.7. Complejidad:....36
4.7.1. Complejidad temporal:....36
4.7.2. Complejidad espacial:....37
5. Ejemplo de ejecución y seguimiento:....38
6. Mediciones:....39
7. Conclusión:....41
**Problema 4 - Programación dinámica:....42**
1. Introducción:....42
2. Definición del problema:....42
3. Análisis del problema:....42
3.1. El desafío de los cortes:....42
3.2. Estrategia de resolución:....43
4. Algoritmo:....43
4.1. Formulación matemática y Ecuación de Recurrencia:....43
4.2. Demostración de optimalidad:....44
4.2.1. Subestructura Óptima:....44
4.2.2. Subproblemas Superpuestos y Memoización:....44
4.3. Pseudocódigo:....45
4.4. Implementación:....46
4.5. Complejidad:....46
4.5.1. Complejidad Temporal:....47
4.5.3. Complejidad Espacial (auxiliar):....47
5. Ejemplo de ejecución y seguimiento:....48
5.1. Paso 1: Matriz de Palíndromos....48
5.2. Paso 2: Arreglo de Cortes....48
5.3. Resultado Final:....49
6. Mediciones:....49

6.1. Metodología de pruebas:.....49
6.2. Tabla de resultados:..... 49
6.3. Gráfico comparativo:..... 50
7. Conclusión:.....51

# Problema 1 - División y conquista:

## 1. Introducción:

En la presente sección se desarrolla el análisis y la resolución del problema de la ”Moneda Falsa”. Se propone un algoritmo basado en el paradigma de División y Conquista para identificar de manera óptima una única moneda defectuosa (más liviana) dentro de un conjunto de n monedas idénticas demostrando la correctitud de la solución y analizando su complejidad computacional. Adicionalmente, se incluyen ejemplos y casos de prueba que permiten validar empıricamente tanto la optimalidad del resultado como el rendimiento estimado del algoritmo.

## 2. Definición del problema:

Se dispone de un conjunto de n monedas de apariencia idéntica. Se sabe con certeza que exactamente una de estas monedas es falsa y que su peso es estrictamente menor al de las n-1 monedas legítimas restantes. La única operación permitida para diferenciar las monedas es utilizar una balanza de platillos que permite comparar el peso de dos subconjuntos a la vez.
El objetivo es diseñar un algoritmo que identifique inequívocamente la moneda falsa minimizando la cantidad de comparaciones (pesadas) requeridas, sin recurrir en ningún momento al ordenamiento del conjunto por peso.

## 3. Análisis del problema:

El problema de identificar una moneda defectuosa sugiere un enfoque de búsqueda por eliminación. A nivel algorítmico, dado que se desconoce el peso nominal de una moneda legítima, la única forma de detectar la anomalía es mediante la comparación de las sumas totales de pesos de distintos subconjuntos.

### 3.1. El enfoque clásico y su caso de falla:

Una primera aproximación de División y Conquista consistiría en dividir el conjunto de $n$ monedas en dos mitades. Sin embargo, este enfoque ingenuo presenta un caso de falla crítico cuando n es impar.
Supongamos un conjunto de $n = 5$ monedas. Si lo dividimos en dos "mitades" asimétricas, obtendremos un conjunto A de 3 monedas y un conjunto B de 2 monedas. Al sumar los pesos crudos para compararlos, el conjunto A siempre sumará un valor mayor que el conjunto B, independientemente de dónde se encuentre la moneda falsa. La cardinalidad mayor del conjunto A enmascara por completo el menor peso de la moneda defectuosa. En este escenario, el algoritmo se vuelve incapaz de decidir con certeza qué subconjunto continuar explorando.

Para subsanar este caso borde, surge una restricción matemática ineludible: solo se pueden comparar sumas de subconjuntos que posean exactamente la misma cantidad de elementos. Podríamos adaptar el enfoque de mitades comparando $n/2$ contra $n/2$ y dejando una moneda fuera, pero esto limitaría la reducción del espacio de búsqueda a la mitad en cada paso.

### 3.2. La estrategia óptima: División en tercios:

El enfoque de mitades no solo es propenso a errores en su versión ingenua, sino que desaprovecha un estado lógico de la comparación: la igualdad. Al comparar dos subconjuntos de igual tamaño, existen tres resultados posibles: el primero es menor, el segundo es menor, o ambos son exactamente iguales.
La estrategia óptima para capitalizar esto consiste en dividir las n monedas en tres subconjuntos de tamaños lo más equitativos posible, y comparar las sumas de solo dos de ellos (que obligatoriamente deben tener el mismo tamaño).
Al hacerlo, la deducción es unívoca:

- Si el primer subconjunto suma menos, la moneda falsa se encuentra allí.
- Si el segundo subconjunto suma menos, la moneda falsa está allí.
- Si ambas sumas son idénticas, la moneda falsa pertenece inevitablemente al tercer subconjunto (el cual no participó de la suma).

Este diseño no solo sortea el problema de las cardinalidades impares manteniendo siempre el balance, sino que permite reducir el espacio de búsqueda a un tercio de su tamaño original en cada iteración. De esta forma, se maximiza la entropía de la información obtenida en cada comparación, garantizando una convergencia asintóticamente más rápida hacia la solución.

## 4. Algoritmo:

En base al análisis previo, se propone un algoritmo recursivo bajo el paradigma de División y Conquista estructurado de forma Top-Down. Para su implementación computacional, la entrada del programa se modelará como un arreglo de números enteros (o flotantes), donde cada elemento representa el peso individual de una moneda. Bajo este modelo, la acción física de colocar monedas en una balanza se traduce algorítmicamente en calcular y comparar las sumatorias de pesos correspondientes a distintos sub-arreglos.
El algoritmo toma el arreglo inicial de monedas y lo divide en tres subconjuntos balanceados, realizando una única comparación de sumas por cada llamada recursiva para descartar dos tercios del problema, hasta aislar la moneda defectuosa en el caso base.

### 4.1. Formulación matemática:

Sea $M = \{m_1, m_2, \cdots, m_n\}$ el conjunto de $n$ monedas, donde cada $m_i$ representa el peso de una moneda. Definimos la condición del problema tal que:

- $m_i = P$ para las $n - 1$monedas legítimas
- $m_f < P$para la única moneda falsa $m_f \in M$

El objetivo es hallar $m_f$ minimizando el número de comparaciones. En cada paso, el algoritmo divide el conjunto M en tres subconjuntos disjuntos A, B y C (tal que $A \cup B \cup C = M$. Se define un tamaño base $k = [n/3]$, asignando equitativamente $|A| = k$, $|B| = k$, y dejando el resto en $|C| = n - 2k$
La operación de comparación evalúa la diferencia de las sumatorias de los pesos totales entre los dos subconjuntos de igual tamaño (A y B):

$$\Delta(A, B) = \sum_{m \in A} m - \sum_{m \in B} m$$

La función de transición de nuestro algoritmo evalúa esta diferencia para aislar unívocamente el subconjunto que contiene a $m_f$:

- Si $\Delta(A, B) < 0 \Rightarrow m_f \in A$
- Si $\Delta(A, B) > 0 \Rightarrow m_f \in B$
- Si $\Delta(A, B) = 0 \Rightarrow m_f \in C$

## 4.2. Demostración de optimalidad:

Para demostrar que el algoritmo propuesto es óptimo respecto a la cantidad de operaciones, podemos analizar el problema desde la perspectiva de la cantidad de información y los estados posibles.
El objetivo del algoritmo es aislar unívocamente la moneda falsa dentro de un conjunto de $n$ monedas. Inicialmente, al carecer de información, cualquiera de las $n$ monedas podría ser la defectuosa. Esto significa que el problema tiene exactamente $n$ "estados" o soluciones posibles.
Por otro lado, debemos analizar nuestra herramienta de extracción de información: la "pesada" (comparación de dos sumas). Cada comparación arroja obligatoriamente uno de tres resultados lógicos: el primer subconjunto suma menos, el segundo suma menos, o ambos suman lo mismo.

- Una (1) pesada permite distinguir, como máximo, entre 3 estados.
- Dos (2) pesadas permiten distinguir entre $3 \times 3 = 9$ estados.
- En general, realizar $k$ pesadas permite generar un máximo de $3^k$ combinaciones de resultados distintos.

Para que un algoritmo garantice hallar la moneda falsa en el peor de los casos, la cantidad de resultados distintos que puede generar mediante sus $k$ pesadas debe ser estrictamente mayor o igual a la cantidad de monedas $n$. Matemáticamente, esta condición de suficiencia se expresa como:

$$3^k \geq n$$

Aplicando el logaritmo en base 3 a ambos lados de la inecuación, obtenemos la cota inferior del problema:

$$k > log_3(n)$$

Esta demostración prueba que es matemáticamente imposible resolver el problema para $n$ monedas realizando menos de $[log_3(n)]$ comparaciones. Dado que nuestro algoritmo reduce el espacio de búsqueda a un tercio de su tamaño en cada iteración ($n/3$), el número

máximo de llamadas recursivas (y por ende, de pesadas) es exactamente $[log_3(n)]$. Al igualar la cota inferior teórica, se confirma formalmente la optimalidad de la solución.

## 4.3. Pseudocódigo:

El algoritmo se diseña utilizando paso de parámetros por índices (inicio y fin). Esta decisión de diseño es crítica para evitar la copia redundante de sub-arreglos en la memoria durante las llamadas recursivas, manteniendo así una complejidad espacial óptima.
Se asume la existencia de una función auxiliar Sumar(M, i, j) que calcula la sumatoria de los elementos del arreglo M desde el índice i hasta el índice j inclusive. Esta función representa computacionalmente el acto de colocar un grupo de monedas en un platillo de la balanza.

```
// Función principal que inicializa la recursión
Función BuscarMonedaFalsa(M):
   Retornar EncontrarMonedaFalsaRecursivo(M, 0, Longitud(M) - 1)

// Función recursiva de División y Conquista
Función EncontrarMonedaFalsaRecursivo(M, inicio, fin):
   // 1. Caso Base: El sub-arreglo tiene exactamente 1 elemento
   // Si solo queda una moneda, inevitablemente es la defectuosa
   n = fin - inicio + 1
   Si n == 1:
      Retornar inicio

   // 2. División: Calcular el tamaño de los subconjuntos
   // k representa el tamaño de los grupos A y B
   k_base = Entero(n / 3)
   resto = n MOD 3

   // Ajustamos el tamaño de A y B (k) para mantener el balance en la balanza
   // Si sobran 2 elementos, distribuimos uno a A y uno a B para igualarlos
   Si resto == 2:
      k = k_base + 1
   Sino:
      k = k_base

   // Definir los límites (índices) de los subconjuntos A y B
   // El grupo A va desde 'inicio' hasta 'inicio + k - 1'
   fin_A = inicio + k - 1

   // El grupo B empieza inmediatamente después de A y tiene el mismo tamaño
   inicio_B = inicio + k
   fin_B = inicio + 2*k - 1

   // El grupo C es el remanente implícito (desde 'inicio + 2*k' hasta 'fin')

   // 3. "Pesada": Comparación de sumatorias
   suma_A = Sumar(M, inicio, fin_A)
   suma_B = Sumar(M, inicio_B, fin_B)
```

```
// 4. Conquista: Evaluar el resultado y realizar la llamada recursiva
Si suma_A < suma_B:
   // El platillo A es más liviano -> La moneda falsa está en A
   Retornar EncontrarMonedaFalsaRecursivo(M, inicio, fin_A)

Sino, Si suma_B < suma_A:
   // El platillo B es más liviano -> La moneda falsa está en B
   Retornar EncontrarMonedaFalsaRecursivo(M, inicio_B, fin_B)

Sino:
   // Ambos platillos pesan lo mismo -> La moneda falsa está en C
   inicio_C = inicio + 2*k
   Retornar EncontrarMonedaFalsaRecursivo(M, inicio_C, fin)
```

## 4.4. Implementación:

```
def sumar_subarreglo(M: list, inicio: int, fin: int) -> float:
   """
   Calcula la suma de los pesos en el subarreglo definido por [inicio, fin]
inclusive.
   Itera sobre índices para evitar la copia de arreglos en memoria (slicing).
   Equivale algorítmicamente a colocar las monedas en la balanza.
   """
   suma = 0
   for i in range(inicio, fin + 1):
      suma += M[i]
   return suma

def encontrar_moneda_falsa_recursivo(M: list, inicio: int, fin: int) -> int:
   """
   Función recursiva Top-Down que halla el índice de la moneda falsa.
   """
   # 1. Caso Base: Si queda una sola moneda, es la defectuosa
   n = fin - inicio + 1
   if n == 1:
      return inicio

   # 2. División: Cálculo de subconjuntos
   k = n // 3
   resto = n % 3

   # Ajuste para mantener el equilibrio de cardinalidad
   if resto == 2:
      k += 1

   # Definición explícita de fronteras (índices)
   fin_A = inicio + k - 1
   inicio _B = inicio + k
```

```
    fin_B = inicio + 2 * k - 1

    # 3. "Pesada": Comparación explícita de sumatorias
    suma_A = sumar_subarreglo(M, inicio, fin_A)
    suma_B = sumar_subarreglo(M, inicio_B, fin_B)

    # 4. Conquista y recursión
    if suma_A < suma_B:
        return encontrar_moneda_falsa_recursivo(M, inicio, fin_A)
    elif suma_B < suma_A:
        return encontrar_moneda_falsa_recursivo(M, inicio_B, fin_B)
    else:
        # La moneda defectuosa está en el grupo C (resto del arreglo)
        inicio_C = inicio + 2 * k
        return encontrar_moneda_falsa_recursivo(M, inicio_C, fin)

def buscar_moneda_falsa(M: list) -> int:
    """
    Función principal que inicializa la búsqueda.
    Retorna el índice donde se encuentra la moneda más liviana.
    """
    if not M:
        raise ValueError("El conjunto de monedas no puede estar vacío.")

    return encontrar_moneda_falsa_recursivo(M, 0, len(M) - 1)
```

## 4.5. Complejidad:

El análisis de complejidad de este algoritmo requiere una distinción rigurosa entre el modelo teórico del problema (la cantidad de pesadas) y su implementación computacional real, resolviendo así la interrogante planteada en la definición del problema.

### 4.5.1. Complejidad Temporal:

Para determinar el tiempo de ejecución, debemos analizar el costo de las operaciones ejecutadas. En cada nivel de recursión:

1. Se calculan los índices para dividir el arreglo en 3 subconjuntos en tiempo constante $O(1)$
2. Se calculan las sumas de los subconjuntos $A$ y $B$ utilizando la función sumar_subarreglo. Dado que los grupos $A$ y $B$ abarcan aproximadamente dos tercios del arreglo actual, la iteración realiza $\frac{2}{3}n$ sumas. Sumar $K$ elementos en un arreglo requiere tiempo proporcional a $K$, por lo que el costo de esta etapa es lineal $O(n)$.
3. Se realiza una (1) única llamada recursiva sobre el subconjunto que contiene la moneda falsa, cuyo tamaño es, en el peor de los casos, $[n/3]$.

La ecuación de recurrencia que modela el tiempo de ejecución $T(n)$ es:

$$T(n) = T(n/3) + O(n)$$

Para resolverla, aplicamos el Teorema Maestro. Identificamos los parámetros $a = 1$ (una llamada recursiva) y $b = 3$ (el problema se divide en tercios). Calculamos el exponente crítico $c = log_b(a) = log_3(1) = 0$.

Dado que el costo de combinación $f(n) = O(n)$ es asintóticamente mayor que $n^{log_3(1)}$(es decir, $O(n^1) > O(n^0)$, nos encontramos en el Caso 3 del Teorema Maestro. El término recursivo es dominado por el costo de iterar para sumar, resultando en un tiempo de ejecución polinomial:

$$T(n) = O(n)$$

### 4.5.2. Análisis de la Cantidad de Pesadas:

Si modelamos la complejidad enfocándonos exclusivamente en la cantidad de "pesadas" $P(n)$, asumimos a la balanza como un "oráculo" de costo constante. Bajo este modelo puramente teórico, sumar los pesos de un platillo cuesta $O(1)$.La ecuación de recurrencia cambia a:

$$P(n) = P(n/3) + O(1)$$

Aplicando el Teorema Maestro $(a = 1,\ b = 3,\ c = 0)$, dado que $f(n) = O(n^0)$ es igual a $n^{log_3(1)}$, nos encontramos en el Caso 2. La cantidad de operaciones de balanza crece logarítmicamente:

$$P(n) = O(log_3 n)$$

### 4.5.3. Complejidad Espacial(auxiliar):

Gracias a la decisión de diseño en la implementación de utilizar exclusivamente índices aritméticos en lugar de slicing nativo (como $M[i:j]$), se evitó la duplicación y creación de nuevos arreglos en memoria durante cada llamada recursiva. El arreglo original de entrada se mantiene estático en el Heap.
El consumo de memoria adicional proviene exclusivamente de la pila de llamadas del sistema (Call Stack) generada por la recursión. Dado que el espacio de búsqueda se reduce a un tercio en cada iteración, la profundidad máxima del árbol de recursión es equivalente a la cantidad máxima de pesadas. Por lo tanto, la complejidad espacial del algoritmo es:

$$S(n) = O(log_3 n)$$

## 4.6. Relación entre Tiempo de Ejecución y Cantidad de Pesadas

¿Es correcto vincular el tiempo de ejecución con la cantidad de pesadas?
No de forma directa. En un modelo estrictamente teórico y físico, una balanza de platillos actúa como una herramienta de evaluación paralela: comparar el peso de 10 monedas o de 10.000 monedas toma un único paso físico de costo constante $O(1)$. Si operamos bajo este modelo ideal, sí sería correcto vincularlos, y el tiempo de ejecución sería directamente proporcional a la cantidad de pesadas, es decir, $O(log_3 n)$.

Sin embargo, en nuestro caso, la "pesada" se simula algorítmicamente calculando la suma de los pesos individuales de los subarreglos. Sumar un grupo de $k$ elementos requiere iterar sobre cada uno de ellos, lo que conlleva un costo lineal $O(n)$.

Por lo tanto, se produce una divergencia entre la lógica y la ejecución: aunque el algoritmo minimiza exitosamente la cantidad de pesadas, el esfuerzo computacional para realizar cada una de esas pesadas es dominante. Esto degrada el tiempo de ejecución real a una complejidad estrictamente lineal $O(n)$, tal como lo confirman las validaciones empíricas y los gráficos de ajuste del presente informe.

# 5. Ejemplo de ejecución y seguimiento:

Para validar empíricamente la lógica de partición y las llamadas recursivas, realizaremos el seguimiento paso a paso del algoritmo propuesto.

Supongamos un conjunto inicial de $n = 8$ monedas. Para el modelado numérico, asumiremos que las monedas legítimas tienen un peso $P = 10$, y la moneda defectuosa tiene un peso de $9$. La moneda falsa se ubicará arbitrariamente en el índice 3.

El arreglo de entrada $M$ será:

$$M = [10,\ 10,\ 10,\ 9,\ 10,\ 10,\ 10,\ 10]$$

Iniciamos la ejecución llamando a la función principal buscar_moneda_falsa(M), la cual desencadena la primera llamada recursiva:

**Nivel de recursión 0 (Inicial)**

- **Parámetros recibidos:** $inicio = 0, fin = 7$
- **Tamaño actual:** $n = 7 - 0 + 1 = 8$. (No es caso base).
- **Cálculo de particiones:** $k = 8 // 3 = 2$
  - $resto = 8 \% 3 = 2$
  - Como el resto es 2, se ajusta $k = 3$
- **Definición de índices y subconjuntos:**
  - **Grupo A:** de índice 0 a 2. → $[10,\ 10,\ 10]$
  - **Grupo B:** de índice 3 a 5. → $[9,\ 10,\ 10]$
  - **Grupo C (implícito):** de índice 6 a 7. → $[10,\ 10]$
- **Operación de Pesada:**
  - suma_A = sumar_subarreglo(M, 0, 2) = 30
  - suma_B = sumar_subarreglo(M, 3, 5) = 29
- **Evaluación y Conquista:**
  - Como suma_B < suma_A (29 < 30), se deduce que la moneda falsa está en el platillo B.
  - Se descartan los grupos A y C (5 monedas en total).
  - Se ejecuta la llamada recursiva sobre el Grupo B.

**Nivel de recursión 1**

- **Parámetros recibidos:** $inicio = 3, fin = 5$
- **Tamaño actual:** $n = 5 - 3 + 1 = 3$. (No es caso base).
- **Cálculo de particiones:** $k = 3 // 3 = 1$
  - $resto = 3 \% 3 = 0$
  - Como el resto no es 2, no hay ajuste: $k = 1$
- **Definición de índices y subconjuntos:**
  - **Grupo A:** de índice 3 a 3. → $[9]$
  - **Grupo B:** de índice 4 a 4. → $[10]$
  - **Grupo C (implícito):** de índice 5 a 5. → $[10]$

- **Operación de Pesada:**
  - suma_A = sumar_subarreglo(M, 3, 3) = 9
  - suma_B = sumar_subarreglo(M, 4, 4) = 10
- **Evaluación y Conquista:**
  - Como suma_A < suma_B (9 < 10), se deduce que la moneda falsa está en el platillo A.
  - Se descartan los grupos B y C.
  - Se ejecuta la llamada recursiva sobre el Grupo A.

**Nivel de recursión 2 (Caso Base)**

- **Parámetros recibidos:** $inicio = 3, fin = 3$
- **Tamaño actual:** $n = 3 - 3 + 1 = 1$.
- **Evaluación:** Al cumplirse $n = 1$, se ha alcanzado el caso base. El algoritmo aísla exitosamente el elemento defectuoso.
- **Retorno:** Se retorna el índice $3$.

**Conclusión del seguimiento:**

El algoritmo logró identificar correctamente la posición de la moneda falsa (índice 3) en un conjunto de 8 elementos realizando únicamente **2 pesadas**. Si verificamos contra nuestra cota teórica del análisis de complejidad $[log_3(8)] = [1.89] \approx 2$, comprobamos que el comportamiento empírico coincide exactamente con la optimalidad matemática demostrada.

# 6. Mediciones:

Para verificar el correcto funcionamiento del algoritmo y confirmar que siempre proporciona la solución óptima, se realizaron pruebas automatizadas utilizando distintos conjuntos de datos generados aleatoriamente. Siguiendo las exigencias del problema, se instanciaron arreglos de tamaño $n = \{100, 500, 1000, 5000, 10000, 50000\}$, asignando el peso defectuoso a un índice aleatorio en cada caso.

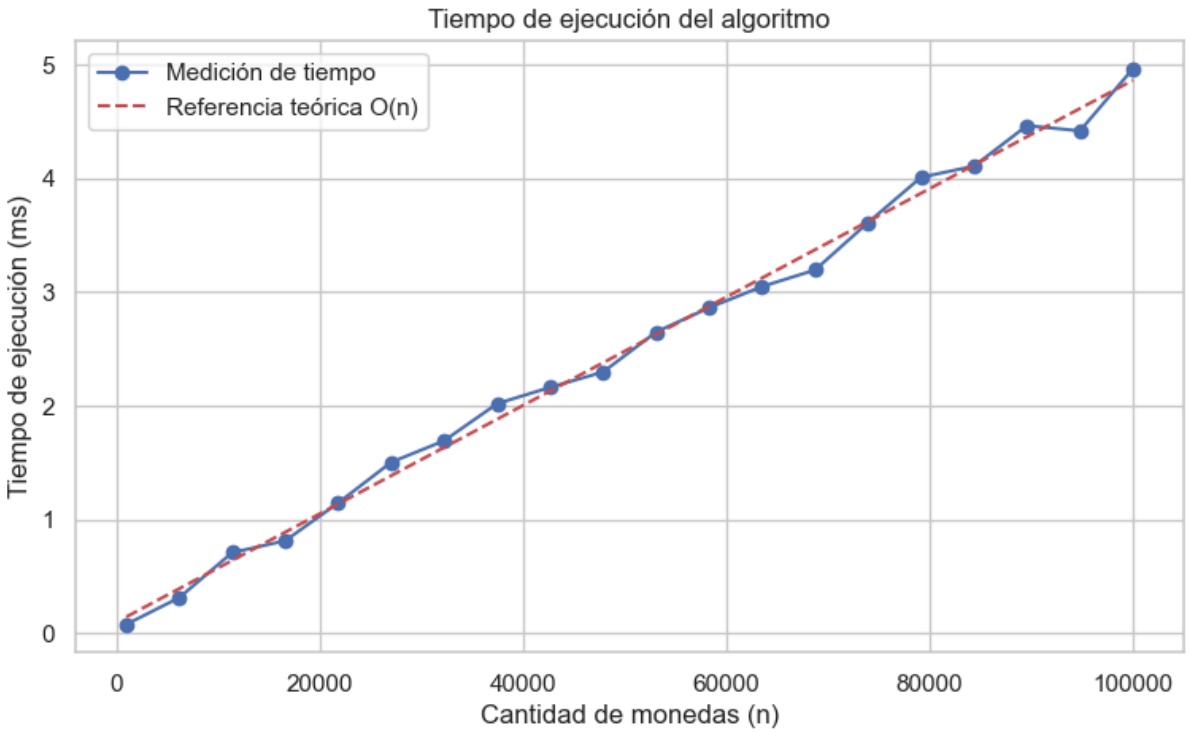
Se presentan los resultados obtenidos en el siguiente cuadro, donde se incluye la posición generada (esperada), el resultado devuelto por el algoritmo, y la cantidad de operaciones de "pesada" (comparaciones de suma) ejecutadas:

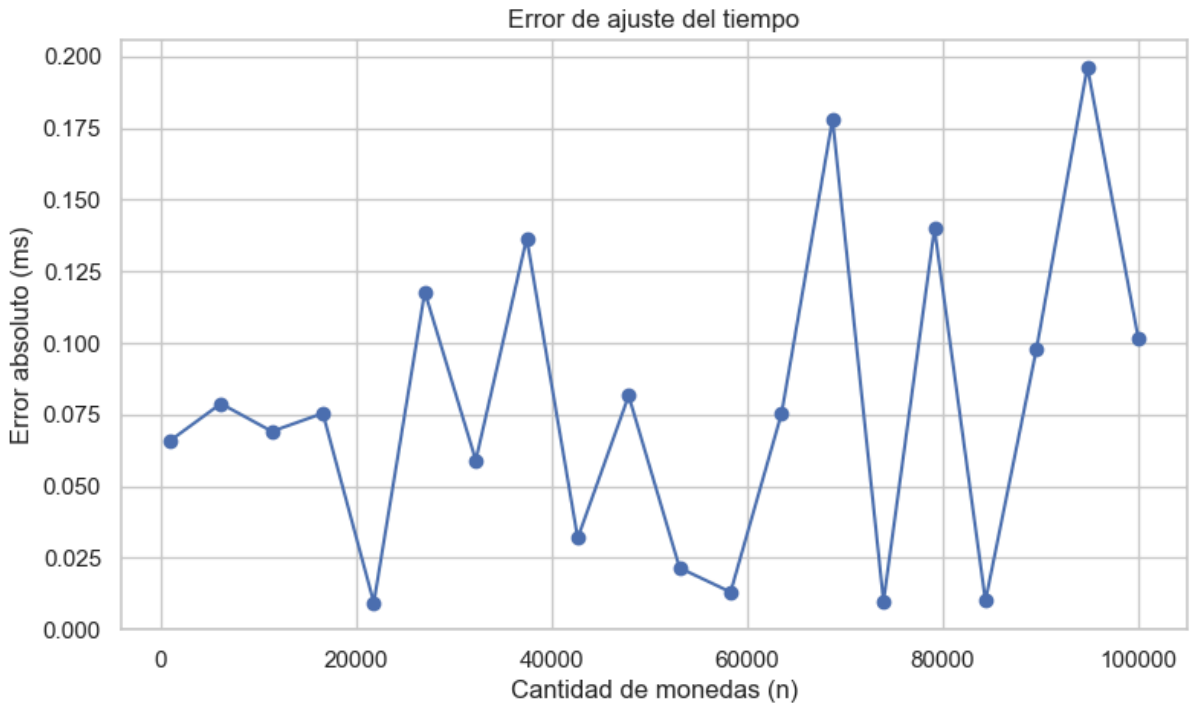
| Tamaño del conjunto $(n)$ | Índice Esperado | Índice Obtenido | Pesadas Ejecutadas | Cota Teórica $log_3(n)$ |
|---|---|---|---|---|
| 100 | 33 | 33 | 4 | $\approx 4.19$ |
| 500 | 166 | 166 | 6 | $\approx 5.66$ |
| 1000 | 333 | 333 | 6 | $\approx 6.29$ |
| 5000 | 1666 | 1666 | 8 | $\approx 7.75$ |
| 10000 | 3333 | 3333 | 9 | $\approx 8.38$ |
| 50000 | 16666 | 16666 | 10 | $\approx 9.85$ |

En todos los casos, podemos ver que se obtiene el índice esperado tras la ejecución, lo que demuestra la correctitud de la lógica de partición. Adicionalmente, se observa empíricamente que la cantidad de pesadas ejecutadas coincide con la cota teórica calculada en la demostración de optimalidad.
Por cuestiones de espacio no se adjunta en este documento el detalle de los arreglos completos ni la traza de cada iteración, pero en el archivo, dentro de la carpeta problema_1, se encuentra el archivo results.txt que incluye el volcado completo de las pruebas.

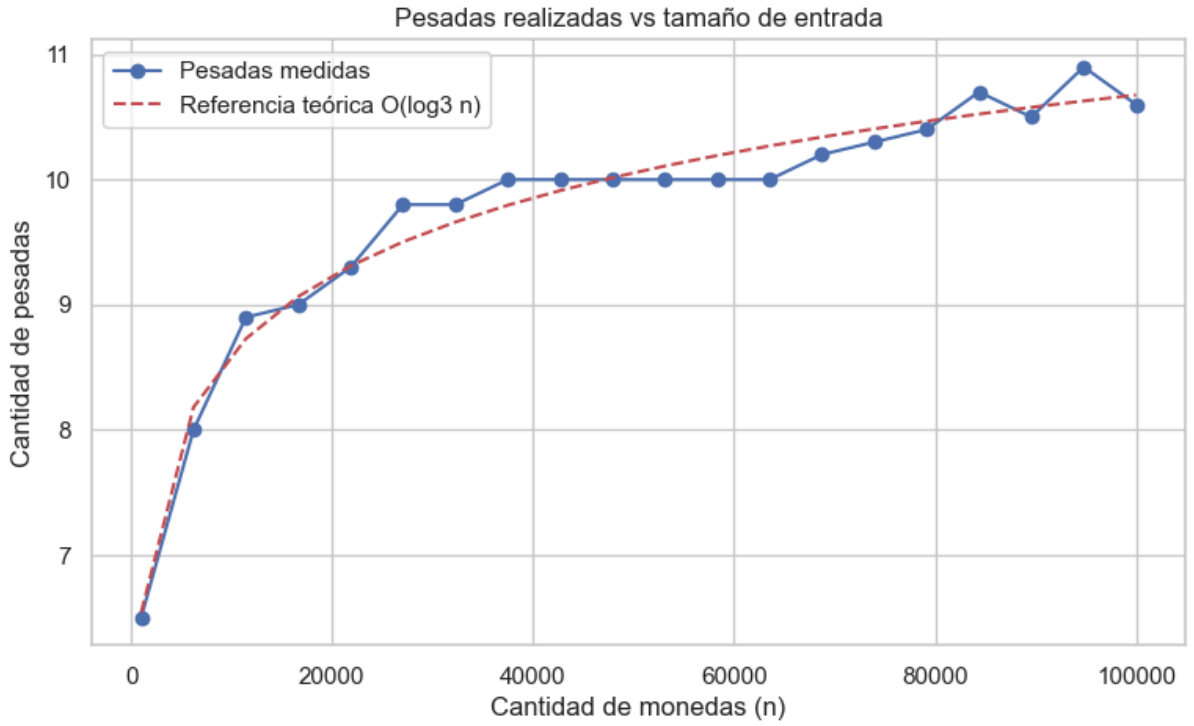
## 6.1. Validaciones empíricas

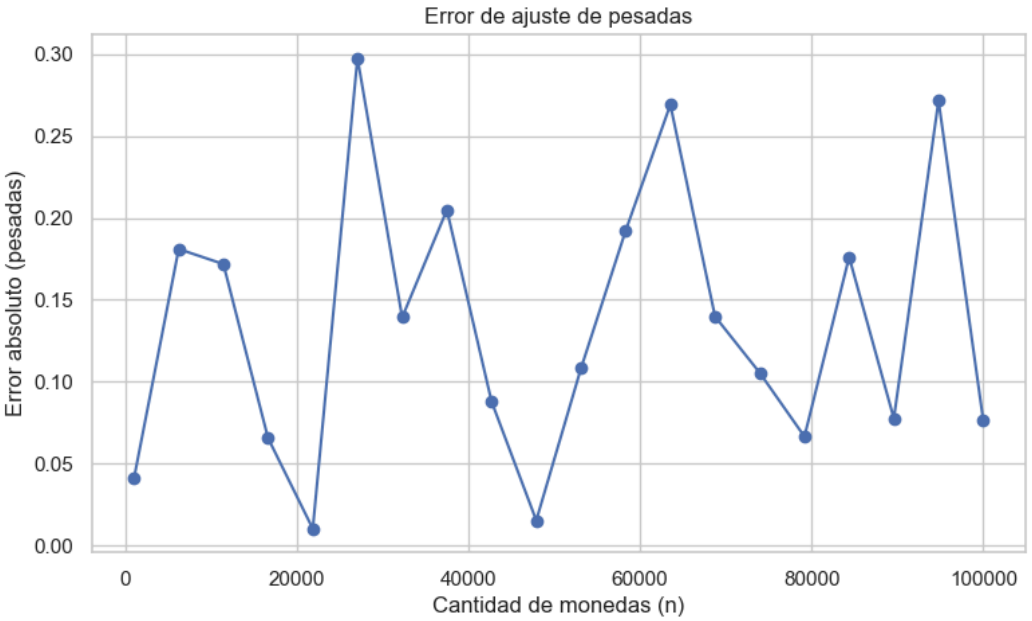
Se realizaron mediciones exhaustivas utilizando diferentes cantidades de monedas. Para simular fielmente el escenario del problema, en cada instancia se instanció un arreglo numérico y se utilizó la biblioteca random para posicionar la única moneda defectuosa (de menor peso) en un índice elegido aleatoriamente.
Para evaluar el algoritmo bajo un volumen de datos significativo, cada conjunto de pruebas contiene arreglos con tamaños $n$ que varían linealmente desde 1.000 hasta 100.000 elementos, distribuidos uniformemente en 20 puntos de medición. En cada uno de estos puntos se registraron dos variables independientes: el tiempo de ejecución computacional y la cantidad de decisiones o "pesadas" realizadas.
Posteriormente, se realizó un ajuste por mínimos cuadrados utilizando la biblioteca scipy y se calculó el error de ajuste para contrastar los resultados empíricos contra las complejidades teóricas demostradas.






Como se observa, la curva exhibe un crecimiento estrictamente lineal $O(n)$. El ajuste por mínimos cuadrados demuestra que, el tiempo está dominado por la operación de sumatoria (sumar_subarreglo) proporcional a la cantidad de elementos, confirmando de manera concluyente la divergencia entre el costo de la pesada teórica y el tiempo de ejecución secuencial analizada en la sección de complejidad.






Como se observa, la curva que representa la cantidad de pasadas se ajusta perfectamente a un modelo de crecimiento logarítmico $log_3(n)$, respaldado por el bajo error de ajuste de scipy. Esto corrobora empíricamente que el espacio de búsqueda se reduce a un tercio en cada iteración, independientemente de la aleatoriedad en la posición de la moneda falsa

# 7. Conclusión:

En este trabajo se propuso, analizó e implementó un algoritmo bajo el paradigma de División y Conquista para resolver de manera óptima el problema de la identificación de una moneda falsa.

A partir de un razonamiento analítico y matemático, se demostró que el enfoque tradicional de dividir el conjunto de elementos en dos mitades resulta subóptimo y propenso a fallas. Como contrapartida, se comprobó que la partición del espacio de búsqueda en tres subconjuntos balanceados capitaliza al máximo los estados lógicos de la balanza, garantizando asintóticamente la menor cantidad de operaciones posibles.

El análisis de complejidad permitió resolver de manera contundente la trampa teórica planteada en el problema, estableciendo una clara distinción entre el modelo teórico y la implementación real. Mientras que el número de "pesadas" logradas por el diseño algorítmico es estrictamente logarítmico $log_3(n)$, el tiempo de ejecución resulta ser lineal $O(n)$, debido a la necesidad ineludible de iterar sobre los elementos para simular la acción física de la sumatoria de masas.

Las validaciones empíricas realizadas sobre conjuntos de datos generados aleatoriamente, junto con los ajustes por mínimos cuadrados, ratificaron de forma precisa y con errores insignificantes los modelos de complejidad desarrollados.

En conclusión, la solución propuesta se erige como un algoritmo robusto, que respeta una estricta complejidad espacial adicional de $log_3(n)$ al operar exclusivamente con índices in-place, y que resuelve el problema planteado alcanzando la frontera de optimalidad matemática exigida por la restricción de comparaciones.

# Problema 2 - Greedy:

## 1. Introducción:

En la presente sección se abordará la resolución del segundo problema del trabajo práctico, enfocado en el análisis de un código de origen desconocido hallado por un equipo de antropología. Partiendo de la hipótesis de que se trata de un algoritmo que opera sobre grafos, se realizará una labor de ingeniería inversa para identificar su propósito, reconstruir su lógica y analizar su complejidad. Específicamente, se demostrará que el código descubierto pertenece a la familia de los algoritmos Greedy. Estos algoritmos se caracterizan por intentar construir una solución óptima global mediante iteraciones sucesivas, aplicando en cada paso una regla sencilla de selección local según el problema dado, hasta alcanzar dicha solución, si es que la estructura del problema lo permite.

## 2. Definición del problema:

Se tiene un grupo de arqueólogos que encontraron en un yacimiento un código escrito en un lenguaje desconocido que se sabe que trabaja sobre grafos. Se nos pide resolver las siguientes interrogante:

- ¿Qué problema resuelve el código encontrado?
- Algoritmo. ¿Qué algoritmo conocido réplica dicho código?
- Estructuras de datos. ¿Cuál es el uso de las principales estructuras de datos del programa?
- ¿Por qué se puede considerar que es un algoritmo Greedy? ¿Cuál es la regla que se aplica?
- Complejidad temporal y espacial. ¿Cuál es el orden de complejidad temporal y espacial del programa? NOTA: no se requiere graficar, sólo una interpretación teórica.
- Traducción. Replicar el código en Python lo más literalmente posible.

Así mismo, para los datos de prueba se cuenta con la siguiente tabla:

| **Eje** | **Costo** |
|---|---|
| 1, 2 | 66 |
| 2, 3 | 122 |
| 2, 4 | 126 |
| 3, 4 | 80 |
| 3, 5 | 148 |
| 4, 5 | 126 |
| 5, 6 | 49 |
| 6, 7 | 101 |

| | |
|---|---|
| 7, 8 | 69 |
| 7, 9 | 72 |
| 8, 9 | 45 |
| 8, 11 | 56 |
| 11, 10 | 30 |
| 10, 9 | 46 |

Todas las interrogantes serán respondidas a lo largo de las siguientes secciones del problema.

# 3. Análisis del problema:

El código analizado, hallado en el archivo paolo.bas, está escrito en un dialecto clásico del lenguaje BASIC. Al analizar su lógica interna y el flujo de la subrutina principal (líneas 1000 a 1120), se pueden responder de manera contundente los interrogantes planteados

## 3.1 Problema que resuelve y algoritmo replicado

El código resuelve el problema del Camino Mínimo desde un Origen Único en un grafo ponderado, conexo y no dirigido, asumiendo aristas con pesos positivos. Dado un vértice de salida particular V, el programa calcula la distancia de recorrido mínimo para llegar desde V hacia todos y cada uno de los demás vértices del grafo.
El algoritmo clásico que replica exactamente esta porción de código es el Algoritmo de Dijkstra. Esto se evidencia claramente en dos pasos fundamentales de la subrutina 1000:

1. La selección greddy del próximo vértice no visitado con la menor distancia acumulada (líneas 1030 a 1060).
2. La actualización de rutas(línea 1090), donde se actualizan las distancias hacia los vértices vecinos si se encuentra un camino más corto a través del vértice recientemente evaluado.

## 3.2 Estructuras de datos utilizadas:

Tanto en el problema como en la solución planteada del siguiente ítem, se están utilizando las siguientes variables que representan las estructuras de datos del algoritmo:

- COST (N,N): Es una matriz de adyacencia (lista de listas) que representa la matriz de costos, siendo esta la representación del gráfico en cuestión. Las aristas representan el costo de ir del vértice i al vértice j, en donde, en caso de no existir la conexión entre los vértices, se utiliza el 15000 para representar un costo infinito.
- DIST (N): Es una lista que eventualmente almacena las distancias mínimas del vértice seleccionado hacia el resto de los vértices.
- SOL (N): Es una lista que indica con un 0 o con 1 si el vértice actual ya fue visitado.

# 4. Algoritmo:

Como ya se indicó con anterioridad, el algoritmo utilizado en el problema es el algoritmo de Dijkstra para la búsqueda de caminos mínimos para grafos con pesos positivos y acíclicos.
Su algoritmo general es el siguiente:

- Se seleccionan un vértice inicial y un vértice final.
- Para cada vértice que se recorre, se selecciona el vértice que tenga menor peso.
- Por cada vértice seleccionado, se actualizan los vértices vecinos a elegir.
- Se repite este procedimiento hasta llegar al último vértice.

## 4.1. Justificación de la naturaleza Greedy

El algoritmo de Dijkstra cumple con esta premisa porque, en cada iteración del bucle principal, el algoritmo toma una decisión irrevocable: selecciona el vértice que tiene la menor distancia acumulada conocida en ese momento y lo marca como "solucionado".
El algoritmo nunca "vuelve atrás" ni reconsidera si una elección previa fue incorrecta (no hay backtracking). Confía en que, al elegir siempre el nodo más cercano no visitado, eventualmente construirá el árbol de caminos mínimos completo.

## 4.2. Regla de selección Greedy

La regla de selección o "función de selección" se encuentra explícitamente en el bloque de las líneas 1030 a 1060:

$$Seleccionar\ el\ vértice\ j\ tal\ que\ j\ \notin\ SOL\ y\ DIST(j)\ sea\ mínimo$$

Computacionalmente, esto se realiza inicializando una variable centinela $U$ con un valor muy alto (el "infinito" de 15000) y recorriendo el arreglo DIST para encontrar el índice que contiene el valor más bajo entre los nodos que aún tienen un 0 en el arreglo SOL.

## 4.3. Formulación matemática:

Sea el grafo $G = (V, E)$, donde $V$ es el conjunto de vértices y $E$ el conjunto de aristas del grafo. A cada arista $(u, v) \in E$ se le asocia un peso no negativo $w(u, v) \geq 0$.
Sea $s \in V$ el nodo de origen. Definimos la distancia real óptima desde $s$ hasta cualquier nodo $v$ como $\delta(s, v)$. El objetivo del algoritmo es que nuestra estimación de distancia $d(v)$ converja al valor óptimo $\delta(s, v)$.
Inicialmente, las distancias se definen como:

- $d(s) = 0$ (La distancia del origen a sí mismo es nula)
- $d(v) = \infty$ para todo $v \neq s$ (En el código, el ∞ se modela con el valor 15000)

La distancia de un camino $P$ es la sumatoria de los pesos de sus aristas. La distancia óptima matemática es:

$$\delta(s, v) = min \sum_{(i,j) \in P} w(i, j)$$

Para alcanzar esta cota mínima, el algoritmo aplica en cada iteración la función de transición generalizada sobre las aristas adyacentes del nodo evaluado $u$:

$$d(v) = min(d(v),\ d(u) + w(u, v)$$

## 4.2. Demostración de optimalidad:

Para demostrar que la estrategia Greedy de Dijkstra garantiza la solución óptima global, utilizaremos el método de demostración por reducción al absurdo.

**Definiciones previas:**

- $u$: Nodo de origen del grafo
- $v, p$: Nodos del grafo
- $d(v)$: Distancia calculada por el algoritmo en un momento dado
- $o(v)$: Distancia optima real del camino más corto
- $S$: Conjunto de vértices ya visitados y resueltos por el algoritmo
- $w(u, v)$: Peso de la arista entre $u$ y $v$, cumpliendo siempre que $w \geq 0$

**Hipótesis**:

Para cada nodo $u \in S$, su distancia calculada actual $d(u)$ es igual a la distancia más corta $o(u)$.

**Validación por contradicción:**

Supongamos que el algoritmo selecciona y añade un nodo $v$ al conjunto $S$, pero existe un camino aún más corto hacia $v$ que el valor $d(v)$ que acabamos de calcular.

Este camino alternativo más corto debe pasar en algún momento por un nodo que **no** se encuentra incluido en $S$. Llamaremos $u$ a este primer nodo fuera de $S$, y $p$ a su nodo anterior inmediato (el cual sí está incluido en $S$).

- Como $p$ está incluido en $S$, por nuestra hipótesis, su camino más corto es conocido y definitivo. Al procesar $p$, el algoritmo actualizó a su vecino $u$, por lo que la distancia estimada hacia el nodo $u$ es $d(u) \leq d(p) + w(p, u)$.
- Debido a que todos los pesos de las aristas son mayores o iguales a cero, cualquier camino que pase por $u$ para llegar finalmente a $v$ tendrá una longitud total que será, como mínimo, igual o mayor a $d(u)$.
- Por lo tanto, para que este camino alternativo sea estrictamente más corto que la ruta actual de $v$, tendría que cumplirse obligatoriamente que $d(u) < d(v)$.
- **La Contradicción:** Esto no tiene sentido. El algoritmo es Greedy y acaba de elegir procesar a $v$ precisamente porque $d(v)$ era el valor mínimo disponible de toda la lista. Es decir, sabemos con certeza que $d(v) \leq d(u)$.

Si por un lado deducimos que $d(u) < d(v)$, y por el otro la regla del algoritmo nos asegura que $d(v) \leq d(u)$, llegamos a un absurdo lógico. Esto demuestra que no existe ningún nodo $u$ tal que ofrezca un camino más corto, confirmando que el algoritmo de Dijkstra siempre devuelve una solución óptima en grafos con pesos no negativos.

## 4.3. Pseudocódigo:

```
Defino función dijkstra(n: int, v: int, cost: List<List<int>>, sol: List<int>,
dist:List<int>):
   Asignar sol[v] a 1
   Asignar dist[v] a 0
   itero desde 0 hasta n - 1 para la búsqueda de caminos mínimos:
      Declaro la dist_min con 15000
      Declaro u con -1
```

itero desde 0 hasta n y me fijo si la distancia actual es menor a la dist_min y si no es solución:
En caso de que se cumpla lo anterior, actualizo dist_min y u con el valor y el índice correspondiente
Si u es -1, termina
Asigno al sol[u] en 1
itero desde 0 hasta n y actualizo el valor de dist[j] por un valor menor.

Defino función main():
Defino n como el input del usuario y lo muestro por pantalla
Inicializo en 0 la matriz cost[][] y las listas dist[] y sol[]

Realizo un while con don ción de corte en el break en caso de que el usuario ingrese el vértice 0 0
Se asignan los valores del vértice A, B según el input del usuario
Se asignan los costos de dicho vértice en la matriz cost[][]
Se completa con 15000 los vértices que no tengan costos asignados, siendo esto el infinito
Se genera la matriz de costos simétrica
Se ingresa a un bucle while con condición de corte futura según el input del usuario con RES
El usuario ingresa el vértice de salida
Para el vértice elegido, asigna a la lista de dis[] los valores de los costos y se asignan 0 a todos los elementos de sol[]
Se invoca a la función dijkstra
Se muestra por pantalla el resultado y se asigna una respuesta RES según el input del usuario

## 4.4. Implementación:

```
import os

def dijkstra(n, v, cost, sol, dist) :

    sol[v] = 1
    dist[v] = 0
    for _ in range(n - 1) :
        dist_min = 15000
        u = -1
        for j in range(n) :
            if dist[j] <= dist_min and sol[j] == 0 :
                dist_min = dist[j]
                u = j
        if(u == -1) : break
        sol[u] = 1
        for j in range(n) :
            if dist[j] >= (dist[u] + cost[u][j]) : dist[j] = dist[u] + cost[u][j]
    return
```

```
def main() :
   os.system('cls' if os.name == 'nt' else 'clear')

   n = int(input("INGRESE EL NUMERO DE VERTICES: "))
   print(n)
   cost = [[0] * n for _ in range(n)]
   dist = [0] * n
   sol = [0] * n
   A = 1

   # Bucle para ingresar los vertices y sus costos
   while(True):

      print("INGRESE EL CUADRO DE COSTOS (INGRESE 0, 0 PARA TERMINAR)")
      print('')
      A, B = map(int,input("EL EJE: ").split()) #El (0, 0) es para salir

      if(A == 0 and B == 0) :
         break

      cost[A - 1][B - 1] = int(input("COSTO DEL EJE "))

   # Los pares de vertices sin costos se les asigna un costo de 15000 ya que se
consideran inaccesibles.
   for i in range(n):
      for j in range(n) :
         if(cost[i][j] == 0) : cost[i][j] = 15000

   # Genera una matriz simetrica ya que el grafo es dirigido.
   for i in range(n):
      for j in range(i):
         cost[i][j] = cost[j][i]

   # Bucle para realizar Dijkstra y volver al bucle anterior en caso de necesitarlo.
   while(True):
      v = int(input("INGRESE EL VERTICE DE SALIDA ")) - 1
      #Asigna los valores la lista de distancias mínimas del nodo destino v, y su
lista de soluciones en 0.
      for i in range(n) :
         dist[i] = cost[v][i]
         sol[i] = 0

      dijkstra(n, v, cost, sol, dist)
      print("SALIDA", "LLEGADA", "DISTANCIA")
      for i in range(n) :
         if dist[i] < 15000 : print(v + 1, " " ,i + 1, " ", dist[i])
      RES = input("OTRA VEZ? (SI/NO) ")
      if RES == "NO" : break

main()
```

## 4.5. Complejidad:

### 4.1. Complejidad Temporal:

Para el análisis de complejidad temporal del algoritmo propuesto, se debe dividir por secciones para determinar la complejidad de cada una de ellas. Cabe aclarar que se desestiman del análisis las condiciones de corte de los bucles `while` principales ya que están ligadas a la entrada estándar del usuario (I/O).

1. **Inicialización y Limpieza de la Matriz:**
   Para identificar las aristas inexistentes y asignarles el costo "infinito" (15000), se recorre la matriz completa de tamaño $n \times x$. Esto requiere $n^2$ iteraciones exactas con operaciones elementales $O(1)$ en su interior. La complejidad de este bloque es estrictamente $O(n^2)$.
2. **Generación de Simetría:**
   Para asegurar que el grafo sea no dirigido, se recorre la matriz triangular inferior. Los bucles anidados realizan iteraciones proporcionales a $n(n-1)/2$. Asintóticamente, descartando constantes y términos de menor grado, esto representa un esfuerzo $O(n^2)$.
3. **Inicialización de arreglos:**
   La copia de la fila inicial de la matriz hacia el arreglo DIST toma $O(n)$ operaciones.
4. **Función dijkstra:**
   El bucle externo se ejecuta $n-1$ veces $O(n)$.
   Dentro de este bucle, ocurren dos procesos secuenciales:
   - **Búsqueda del mínimo:** Un bucle recorre el arreglo de tamaño $n$ para encontrar el valor menor no visitado. Complejidad local: $O(n)$.
   - **Actualización:** Otro bucle independiente recorre el arreglo de tamaño $n$ para actualizar distancias. Complejidad local: $O(n)$.

   La complejidad de la subrutina está dada por el producto del bucle exterior y la suma de las operaciones interiores:

$$O(n) \times (O(n) + O(n) + O(1)) = O(n^2)$$

En conclusión, la suma de las complejidades de todos los bloques secuenciales está dominada por el término de mayor crecimiento. Por lo tanto, el orden de complejidad temporal del programa completo es

$$T(n) = O(n^2).$$

### 4.2. Complejidad Espacial:

La complejidad espacial del programa está determinada principalmente por las estructuras de datos inicializadas y cuanto espacio ocupan estas en memoria, dimensionadas en función de $n$:

- **Variables simples:** Los punteros e iteradores ocupan un espacio constante $O(1)$.
- **Arreglos Unidimensionales:** Las listas dist y sol almacenan $n$ elementos enteros cada una. El espacio requerido escala linealmente, resultando en una complejidad de $O(n)$.

- **Matriz de Adyacencia:** La estructura cost, modelada como una lista de listas, debe almacenar un elemento por cada posible conexión entre vértices. Contiene $n$ filas con $n$ columnas cada una. Esto requiere alojar $n \times n$ elementos enteros en memoria, resultando en una complejidad de $O(n^2)$.

En conclusión, sumando las asignaciones estáticas, $O(1) + O(n) + O(n^2)$, el factor dominante es la matriz de adyacencia. Por consiguiente, la complejidad espacial del programa escala cuadráticamente:

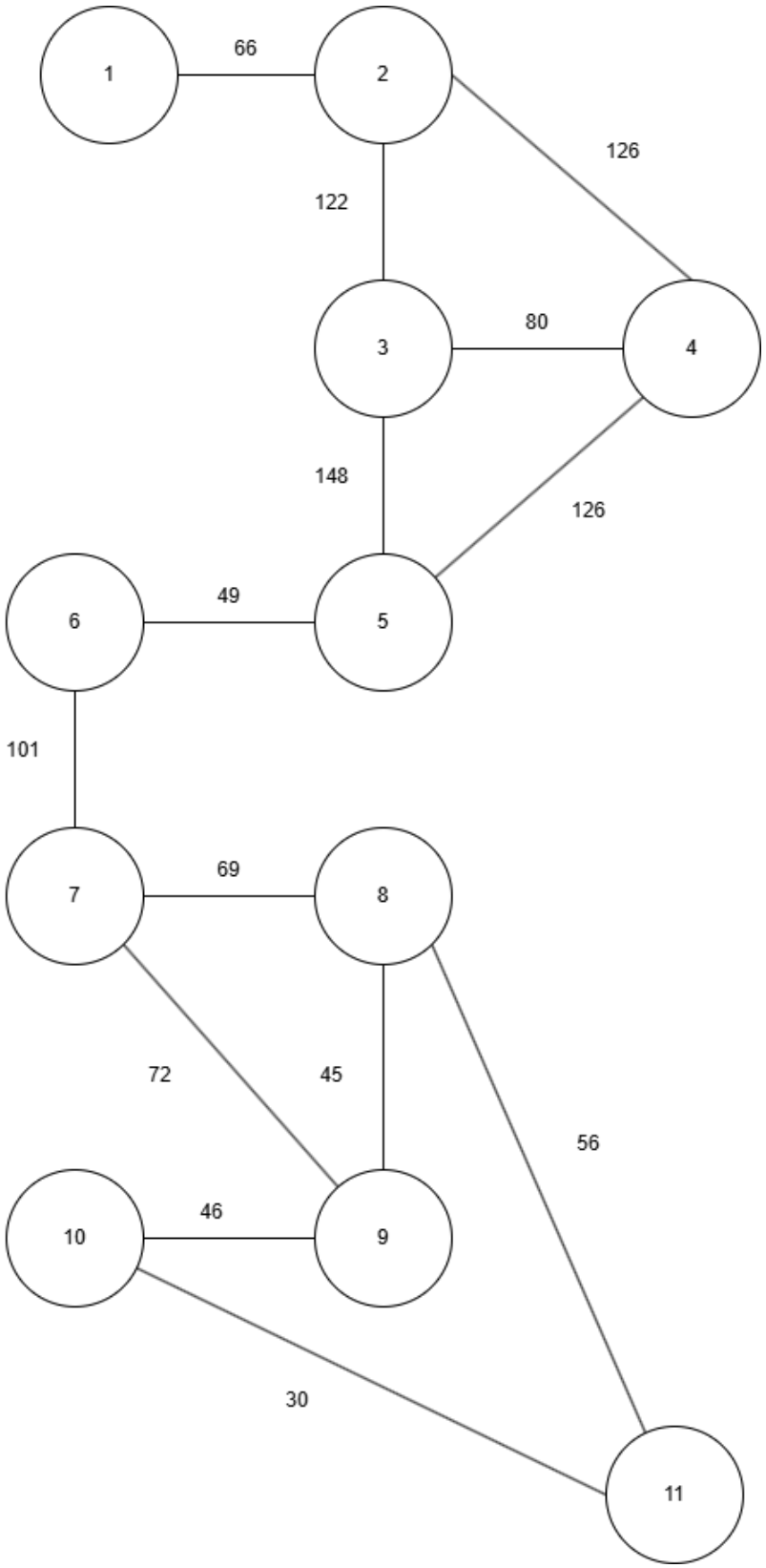
$$S(n) = O(n^2).$$

# 5. Ejemplo de ejecución y seguimiento:

Se tiene un grafo con la siguiente tabla de costos:

| Eje | Costo |
|---|---|
| 1, 2 | 66 |
| 2, 3 | 122 |
| 2, 4 | 126 |
| 3, 4 | 80 |
| 3, 5 | 148 |
| 4, 5 | 126 |
| 5, 6 | 49 |
| 6, 7 | 101 |
| 7, 8 | 69 |
| 7, 9 | 72 |
| 8, 9 | 45 |
| 8, 11 | 56 |
| 11, 10 | 30 |
| 10, 9 | 46 |

Una representación de esta grafo puede ser la siguiente:



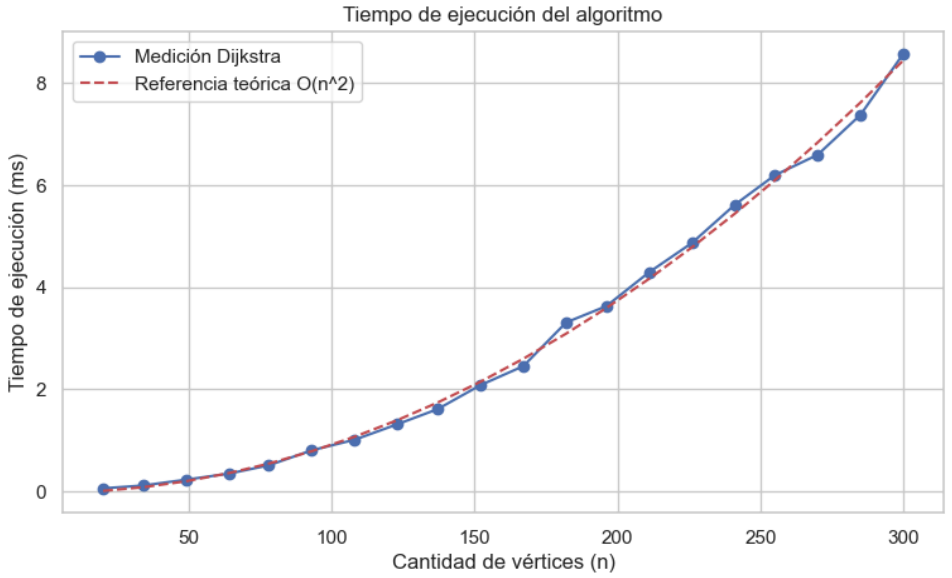
Teniendo una representación visual del grafo, voy a realizar el seguimiento del código con uno de los vértices, por ejemplo, para el vértice 10:

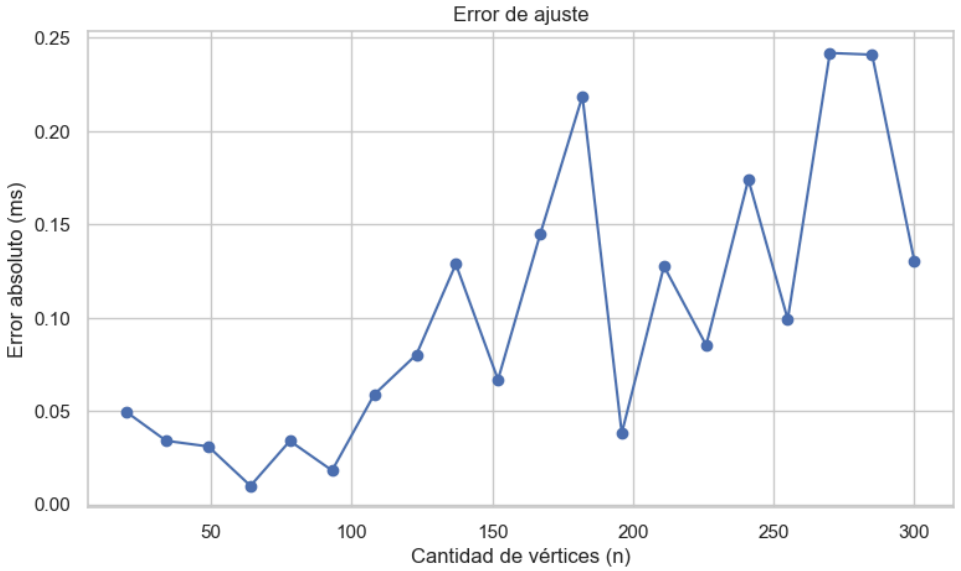
```
n = 11
# Muestra por pantalla (11)
# cost = cost[i][j] = 0 (Indica que cost es una matriz de 11 * 11 con todos 0)
# dist = dist[i] = 0
# sol = sol[i] = 0

# Ingresa al while(True) y se va a repetir n = 11
# Muestra por pantalla ("INGRESE EL CUADRO DE COSTOS (INGRESE 0, 0 PARA
TERMINAR)")
# Iteration 0:
# A, B = input(1, 2)
# cost[0][0] = 66
# El usuario debe ingresar el resto de los vértices y los costos.

# Una vez que se ingresaron todos los valores,  se ejecutarán los bucles para asignar a
cost[i][j] los valores con 15000
# y después se completa la matriz haciéndola simétrica.
# Ahora se ingresa al segundo while(true) que va a finalizar cuando el usuario escriba
END
# v = int(input("INGRESE EL VERTICE DE SALIDA ")) - 1, en el ejemplo el usuario pone
10
# v = 9
# Se completa d[i] que queda como con cost[10, i]
# dist = [
    15000,
    15000,
    15000,
    15000,
    15000,
    15000,
    15000,
    15000,
    46,
    15000,
    30
]
# También se completa sol[i] con 0
# Se invoca a dijkstra(n, v, cost, sol, dist)
# sol[9] = 1
# dist[9] = 0
```

```
# Ahora se van a ir modificando todos los valores de dist[i] hasta el costo mínimo hacia
cada vértice
# Para simplificar, voy a desarrollar el caso de i = 0 (vértice 1)
# Se ingresa al for de rango (0, 13)
# dist_min = 15000
# u = -1
# Se elige el menor dist[j] con sol[j] = 0
# => u = 10
# dist_min = 30
# sol[10] = 1
# Se ingresa al tercer bucle de rango (0, 14), y con d[10] = 30, cost[10][j]
# Ahora, para j = 7 se cumple la condición del algoritmo:
# teniendo dist[j] = dist[7] = 15000 y cost[10][7] = 56
# ==> 15000 >= 30 + 56 ==> dist[7] = 86
# y dist queda:
# dist = [
    15000,
    15000,
    15000,
    15000,
    15000,
    15000,
    15000,
    86,
    46,
    15000,
    30
]
# Este procedimiento se va a realizar para todos los valores de dist, y completando con 1
todos los valores de sol
# Una vez completado dist, simplemente se imprimen los valores por pantalla, teniendo
como resultado:
10   1   586
10   2   520
10   3   416
10   4   394
10   5   268
10   6   219
10   7   118
10   8   86
10   9   46
10   10   0
10   11   30
```

# 6. Mediciones y validaciones empíricas:

Se realizaron mediciones exhaustivas del algoritmo traducido a Python para contrastar su rendimiento empírico real con el análisis de complejidad teórico.
Para simular fielmente el escenario del problema y garantizar la validez topológica de las pruebas, se generaron dinámicamente grafos no dirigidos, conexos y ponderados con costos positivos utilizando la biblioteca random. Para asegurar la conectividad en cada instancia, se construyó primero una cadena base que une secuencialmente todos los vértices, y posteriormente se añadieron aristas aleatorias adicionales evitando duplicados.
Para evaluar el comportamiento asintótico, cada conjunto de pruebas evaluó la cantidad de vértices ($n$) variando linealmente desde 20 hasta 300 elementos, distribuidos uniformemente en 20 puntos de medición. En cada punto se cronometró el tiempo de ejecución en milisegundos. Finalmente, se realizó un ajuste por mínimos cuadrados utilizando la biblioteca scipy y se calculó posteriormente el error absoluto del ajuste contra la complejidad teórica demostrada.



Como se observa claramente en el primer gráfico, la curva de tiempo de ejecución real se acopla con un nivel de precisión altísimo a la curva teórica de referencia $O(n^2)$. El algoritmo exhibe un crecimiento estrictamente parabólico a medida que el volumen de vértices aumenta.
Esta correlación queda demostrada de forma concluyente en el gráfico de error absoluto. Podemos observar que la diferencia máxima entre el tiempo predicho por el modelo teórico y la medición real en todo el rango de pruebas alcanza un tope minúsculo de apenas 0.25 milisegundos. Esta fluctuación insignificante valida empíricamente el análisis realizado en el punto anterior: el uso de la matriz de adyacencia y la búsqueda anidada del valor mínimo dictan invariablemente una complejidad temporal $O(n^2)$.

# 7. Conclusión:

En esta segunda sección del trabajo se analizó y decodificó, mediante ingeniería inversa, un programa bajo el paradigma Greedy. Se logró identificar que el código replica exactamente el Algoritmo de Dijkstra para resolver el problema del Camino Mínimo desde un origen único.
A partir de un razonamiento analítico y matemático, se demostró que el enfoque de realizar elecciones locales irrevocables resulta suficiente para alcanzar el óptimo global. Mediante una demostración por reducción al absurdo, se comprobó que esta regla de selección capitaliza al máximo la estructura del grafo, garantizando la ruta más corta.
El análisis de complejidad permitió establecer de manera contundente el costo computacional del programa. Se demostró que el tiempo de ejecución resulta ser polinomial cuadrático $O(n^2)$, debido a la necesidad de iterar sobre una matriz de adyacencia estática para simular el proceso de actualización de distancias.
Las validaciones empíricas realizadas sobre la posterior traducción del código a Python, junto con los ajustes por mínimos cuadrados, ratificaron de forma precisa y con errores ínfimos el modelo de crecimiento cuadrático desarrollado.
En conclusión, el código descubierto se erige como una solución algorítmica robusta, que respeta una estricta complejidad espacial y temporal de $O(n^2)$ al operar exclusivamente con estructuras contiguas básicas, y que resuelve el problema planteado alcanzando la frontera de optimalidad matemática.

# Problema 3 - Backtracking:

## 1. Introducción:

En la presente sección se abordará la resolución del tercer problema del trabajo práctico, enfocado en la navegación de una aspiradora robot dentro de un entorno cerrado. Para la resolución de este desafío, se diseñará e implementará un algoritmo aplicando la técnica algorítmica de Backtracking. Se demostrará cómo esta estrategia de exploración exhaustiva permite recorrer el espacio de estados del laberinto, construyendo una ruta de manera incremental y retrocediendo estratégicamente cada vez que se detecta un camino sin salida, garantizando así el hallazgo de una solución válida si la topología del laberinto lo permite.

## 2. Definición del problema:

El problema plantea modelar computacionalmente el comportamiento de una aspiradora robot que debe escapar de un laberinto rectangular de dimensiones $m \times n$. Dicho entorno se encuentra codificado en forma matricial, donde las celdas pueden contener los siguientes elementos: “X” (Pared), “S” (Salida), “E” (Posición inicial del robot) y “-” (Camino transitable).
La principal restricción del problema radica en las capacidades del agente. Para encontrar su camino, la aspiradora no tiene una vista global del mapa, sino que está limitada a ejecutar exclusivamente tres operaciones primitivas relativas a su posición y orientación actual:

- Inspeccionar el estado de la celda que tiene inmediatamente adelante (detectando si es pared, camino o salida).
- Avanzar una posición en la dirección que está mirando.
- Girar 90º sobre su propio eje hacia la izquierda.

El objetivo es diseñar un algoritmo que, operando estrictamente bajo este conjunto de movimientos, determine la secuencia de pasos necesarios para que la aspiradora alcance la celda objetivo ("S").

## 3. Análisis del problema:

El problema central consiste en trazar una ruta continua desde una posición inicial (“E”) hasta una salida (“S”), respetando las restricciones de ceguera periférica y movimiento de la aspiradora robot.

### 3.1 Primer enfoque de la solución

Una primera aproximación teórica para resolver esto sería utilizar un enfoque de "Fuerza Bruta". Esta técnica consiste en generar sistemáticamente todas las permutaciones posibles de secuencias de movimiento (combinaciones de avanzar y girar) y, posteriormente, simularlas para verificar si alguna resulta en un camino exitoso hacia la salida.
Si bien este enfoque, por su naturaleza exhaustiva, terminaría encontrando una solución (si existe), resulta computacionalmente inviable e ineficiente. Al no evaluar el entorno de forma reactiva, la Fuerza Bruta no descarta caminos inválidos de manera temprana.

Por ejemplo, supongamos que la posición en frente al punto de entrada (es decir, la que se encuentra inmediatamente delante) es una pared. Este enfoque no descarta caminos inválidos de forma temprana, ya que puede continuar explorando secuencias de movimientos incluso después de haber tomado decisiones que inevitablemente lo llevarán a un estado sin salida.
Más adelante se analizará en profundidad la complejidad de esta técnica, pero es importante destacar que, en el peor de los casos, la cantidad de posibles secuencias de movimientos crece exponencialmente con el tamaño del laberinto.

## 3.2 El enfoque por Backtracking

La técnica de Backtracking se presenta como el paradigma ideal para resolver este problema, ya que se caracteriza por construir una solución de forma incremental y descartar (podar) inmediatamente aquellas soluciones parciales que violan las restricciones del entorno.
El algoritmo capitaliza la herramienta principal de la aspiradora: su sensor frontal. Al "ver lo que tiene adelante", el robot evalúa la viabilidad del siguiente paso en tiempo real. Si detecta una pared ("X") o una celda que ya fue visitada, el algoritmo asume instantáneamente que ninguna solución global puede incluir ese movimiento parcial. En ese momento, se "poda" esa rama del árbol de búsqueda, el robot gira 90° para evaluar una nueva dirección y, si se encuentra en un callejón sin salida, ejecuta un retroceso físico hacia la última bifurcación con opciones inexploradas.
Con esta estrategia se logra reducir drásticamente el espacio de estados a analizar. Al no generar secuencias a ciegas, sino interactuar dinámicamente con el laberinto, el Backtracking garantiza encontrar un camino de escape válido de manera eficiente, o bien concluir de forma determinista que el laberinto no tiene solución si la aspiradora retrocede hasta el punto de entrada ("E") y agota todas sus opciones de giro.

# 4. Algoritmo:

Teniendo en cuenta el análisis previo, se diseña un algoritmo de Backtracking que explora las rutas topológicas desde el punto de entrada de manera incremental. A diferencia de un planteo de Fuerza Bruta tradicional, este algoritmo descarta dinámicamente aquellas ramas de ejecución que derivan en una pared ('X') o en una celda ya visitada.

**La representación del laberinto:**

- X para las paredes
- E para el punto de entrada
- S para la salida
- Los espacios en blanco representan lugares transitables.

**Consideraciones operativas**:

Cuando la aspiradora se encuentra en estado de exploración (aún no ha detectado la salida 'S'), su comportamiento se rige estrictamente por su set de instrucciones:

1. Inspecciona su frente.
2. Si es transitable y no ha sido visitado, avanza y delega la exploración recursivamente.
3. Si choca con una pared, un límite o una celda ya visitada, comienza a girar 90 grados hacia la izquierda para evaluar la siguiente orientación disponible.
4. Si agota sus 4 orientaciones sin éxito, el algoritmo deduce que la celda actual es un callejón sin salida, por lo que retorna falso y obliga al robot a retroceder a la celda anterior.

## 4.1. Formulación matemática:

Sea $M$ una matriz de tamaño $m \times n$ que representa el laberinto.
Buscamos encontrar un camino $C$, definido como un vector ordenado de coordenadas o celdas $C = (V_1, V_2, \cdots, V_k)$ , tal que cumpla estrictamente con las siguientes condiciones:

1. Condición de inicio: $V_1 = "E"$ (El origen)
2. Condición de éxito: $V_k = "S"$ (La salida)
3. Restricción de Adyacencia: $V_i \, y \, V_{i+1}$ deben ser celdas ortogonalmente adyacentes para todo $V_i \in C$
4. Restricción de Obstáculos: $V_i \neq "X"$ para todo $V_i \in C$ (Ninguna celda puede ser una pared).
5. Restricción de Ciclos (Camino Simple): $V_i \neq V_j$ para todo $i \neq j$ (No se permite visitar la misma celda más de una vez para evitar bucles infinitos).

## 4.2. Demostración de correctitud y completitud:

En el análisis formal de este algoritmo, es fundamental destacar que Backtracking no resuelve el problema encontrando el camino más corto, es decir, no garantiza la optimalidad del recorrido. Esto se debe a que el algoritmo prioriza la exploración de una rama (siempre y cuando sea una solución posible), en vez de evaluar qué alternativa promete ser globalmente mejor. Esto es en base a la naturaleza del problema: la aspiradora no es capaz de tomar decisiones inteligentes, simplemente puede ir descartando aquellos caminos que ya exploró y que no tienen solución. Como consecuencia, el camino trazado hacia la salida puede ser notablemente más largo que la ruta óptima teórica.
A pesar de carecer de esta métrica de optimalidad, podemos distinguir dos características fundamentales en nuestra solución:

- **Completitud**: Debido a que el algoritmo evalúa exhaustiva y sistemáticamente el árbol de estados, y su restricción de memoria evita los ciclos infinitos, el sistema te garantiza que siempre encontrará un camino hacia la salida si es que este existe
- **Correctitud**: En caso de que el laberinto sea irresoluble (la salida está bloqueada), el algoritmo podará todas las ramas y retrocederá hasta el punto "E", demostrando que el espacio carece de solución, sin caer en ejecuciones infinitas

## 4.3. Estructuras de datos y control de estado:

Para resolver el problema de manera satisfactoria y evitar ciclos infinitos, el algoritmo requiere instanciar dos estructuras de datos fundamentales para mantener el estado de la exploración:

- Pila de Recorrido (camino): Una estructura dinámica (tipo Pila o Lista) que almacenará las coordenadas de los pasos que conforman la ruta actual. Su función es registrar el rastro de la aspiradora. Crucialmente, esta estructura debe permitir "desapilar" (eliminar) el último elemento insertado cuando el algoritmo detecta un callejón sin salida y necesita retroceder.
- Conjunto de Memoria (visitados): Una estructura de acceso rápido (como un Set o una Matriz booleana) que guardará las coordenadas (x, y) de las celdas por las que la aspiradora ya ha transitado.

**Justificación del conjunto visitados:**
Esta estructura es vital para garantizar la finalización del algoritmo. Supongamos un escenario en el laberinto donde la aspiradora se encuentra en una posición $A$ rodeada de paredes, excepto por un único pasillo a su izquierda. Al avanzar hacia esa celda libre, su única opción de movimiento futuro (si no encuentra la salida) sería girar 180° y volver a $A$. Si la posición $A$ no estuviera marcada como "visitada", el robot volvería a ella, luego volvería al pasillo, y así entraría en un loop infinito (oscilando entre dos baldosas). Al marcar cada celda pisada, el algoritmo detecta que volver sobre sus pasos no es una opción de exploración válida, forzando el retorno.

## 4.4. Arquitectura de funciones y lógica general:

El algoritmo debe mantener un mapeo interno para traducir los movimientos físicos a coordenadas absolutas de la matriz. Para esto, el estado físico del robot se modela con una tupla (x, y, dirección), donde la dirección se representa mediante un valor entero (ej. 0=Norte, 1=Este, 2=Sur, 3=Oeste). La estructura del algoritmo se divide en dos funciones principales que interactúan entre sí:

- **ResolverLaberinto**(): Es la función principal (o "wrapper"). Se encarga de inicializar las estructuras de datos vacías (la pila de camino y el conjunto de visitados), obtener las coordenadas iniciales del robot y lanzar la primera llamada a la función de búsqueda. Retorna el camino calculado o informa si el laberinto no tiene solución.
- **BuscarCamino**(): Es la función recursiva central que implementa el Backtracking. Recibe la posición actual, la pila del camino y el conjunto de visitados. Devuelve Verdadero si desde esa celda pudo encontrar una ruta hacia la salida, o Falso en caso contrario.

**Lógica de ejecución paso a paso:**
Por cada llamada recursiva (cada vez que la aspiradora pisa una baldosa nueva), el algoritmo ejecuta la siguiente secuencia:

- **Registro**: Inmediatamente agrega la posición actual a la pila camino y al conjunto visitados. Aquí radica una de las principales mejoras del Backtracking: al marcar la celda, el robot ya sabe que no debe volver a pisarla, evitando ciclos infinitos.

- **Evaluación del entorno**: Al no tener visión periférica, la aspiradora entra en un bucle de hasta 4 iteraciones donde mira hacia adelante. Si detecta la salida, avanza y el algoritmo frena devolviendo éxito.
- **Avance y Poda**: Si detecta que al frente hay un camino libre y que no está en el conjunto visitados, avanza físicamente y se llama a sí misma recursivamente. Si, por el contrario, detecta una pared o una celda ya visitada, "poda" esa opción instantáneamente y gira 90° a la izquierda para evaluar una nueva dirección.
- **Retroceso** (El Backtrack): Si la aspiradora gira 4 veces (360°) y descubre que ninguna dirección es válida, concluye que está en un callejón sin salida. En este punto, elimina la celda actual de la pila camino (para no dejar "basura" en la ruta final), devuelve Falso, y ejecuta una maniobra de retroceso físico para volver a la baldosa anterior y continuar explorando otras ramas.

## 4.5. Pseudocódigo:

```
// Función principal de inicialización
Función ResolverLaberinto(origen, destino, laberinto):
    camino = nueva Pila()
    visitados = nuevo Conjunto()

    // Se inicia la búsqueda asumiendo la orientación 0 (hacia Arriba)
    Si BuscarCamino(origen, 0, destino, laberinto, visitados, camino) es VERDADERO:
        retornar camino
    Sino:
        retornar NULO

// Función recursiva de Backtracking
Función BuscarCamino(pos_actual, direccion, destino, laberinto, visitados, camino):
    // 1. Registrar el estado espacial (independiente de la dirección)
    visitados.agregar(pos_actual)
    camino.apilar(pos_actual)

    // Condición de éxito: Se alcanzó la celda objetivo
    Si pos_actual == destino:
        retornar VERDADERO

    // 2. Explorar el entorno girando hasta 4 veces
    dir_actual = direccion
    Para iteracion desde 1 hasta 4:
         // Traducir la orientación a coordenadas de la matriz
        pos_siguiente = MirarAdelante(pos_actual, dir_actual)

        // Validar límites del tablero, obstáculos y memoria de estado
        Si (pos_siguiente está dentro de los límites del laberinto) Y
           (laberinto[pos_siguiente] != "X") Y
```

```
        (pos_siguiente NO pertenece a visitados):

         // Llamada recursiva avanzando hacia la celda válida
        Si BuscarCamino(pos_siguiente, dir_actual, destino, laberinto, visitados, camino)
es VERDADERO:
            retornar VERDADERO

      // Si la celda era inválida o la recursión falló, girar 90° a la izquierda
      // Se utiliza aritmética modular para ciclar entre las 4 direcciones (0 a 3)
      dir_actual = (dir_actual + 1) MOD 4

   // 3. BACKTRACKING LÓGICO: Todas las rutas desde esta celda fallaron
   camino.desapilar() // Se elimina la celda de la ruta final
   retornar FALSO

// Función auxiliar: Calcula la coordenada adyacente según la orientación
Función MirarAdelante(posicion, direccion):
   x, y = posicion
   Si direccion == 0: retornar (x - 1, y)   // Arriba
   Si direccion == 1: retornar (x, y - 1)   // Izquierda
   Si direccion == 2: retornar (x + 1, y)   // Abajo
   Si direccion == 3: retornar (x, y + 1)   // Derecha
```

## 4.6. Implementación:

```
# Funcion principal de inicializacion
def resolver_laberinto(entrada: tuple[int, int], salida: tuple[int, int], laberinto: list[list[str]]) ->
list | None:
   visitados = set()
   camino = []

   # Se asume que la aspiradora arranca mirando hacia arriba (direccion 0)
   if buscar_camino(entrada, 0, salida, laberinto, visitados, camino):
      return camino
   return None

# Función recursiva de Backtracking
def buscar_camino(pos_actual, direccion, salida, laberinto, visitados, camino):
   # 1. Registrar estado espacial (independiente de la dirección)
   visitados.add(pos_actual)
   camino.append(pos_actual)

   # Condición de éxito: encontramos la salida
   if pos_actual == salida:
```

```
        return True

    # 2. Explorar el entorno físico girando hasta 4 veces
    dir_actual = direccion
    for _ in range(4):
        pos_sig = mirar_adelante(pos_actual, dir_actual)

        # Si la celda de enfrente está dentro del laberinto, no es pared y NUNCA fue pisada
        if (0 <= pos_sig[0] < len(laberinto) and
            0 <= pos_sig[1] < len(laberinto[0]) and
            laberinto[pos_sig[0]][pos_sig[1]] != "X" and
            pos_sig not in visitados):

            # Llamada recursiva (avanzamos físicamente a esa celda)
            if buscar_camino(pos_sig, dir_actual, salida, laberinto, visitados, camino):
                return True

        # Si había pared, o estaba visitada, o el camino explorado falló (retornó False):
        # Giramos 90 grados a la izquierda para probar el siguiente frente
        dir_actual = (dir_actual + 1) % 4

    # 3. BACKTRACKING LÓGICO: Probamos las 4 direcciones y ninguna sirvió.
    # Desapilamos esta celda porque no forma parte del camino de salida.
    camino.pop()
    return False

# Función auxiliar de sensor (traduce la dirección física a coordenadas de matriz)
# 0: Arriba, 1: Izquierda, 2: Abajo, 3: Derecha
def mirar_adelante(posicion: tuple[int, int], direccion: int) -> tuple[int, int]:
    if direccion == 0:
        return (posicion[0] - 1, posicion[1])
    if direccion == 1:
        return (posicion[0], posicion[1] - 1)
    if direccion == 2:
        return (posicion[0] + 1, posicion[1])
    # direccion == 3
    return (posicion[0], posicion[1] + 1)
```

## 4.7. Complejidad:

### 4.7.1. Complejidad temporal:

Para dimensionar la eficiencia de la solución propuesta, resulta imperativo contrastarla con el costo de un enfoque de Fuerza Bruta ingenuo.

Si resolviéramos este laberinto generando sistemáticamente todas las combinaciones posibles de movimientos sin memoria de estado (sin el conjunto de visitados), el robot podría entrar en ciclos infinitos. Incluso limitando la longitud máxima del camino a $k$ pasos, como en cada baldosa el robot tiene hasta 4 rotaciones posibles, el árbol de expansión tendría un crecimiento exponencial de orden $O(4^k)$.
La implementación mediante Backtracking neutraliza esta explosión combinatoria. La clave de esta optimización radica en el conjunto visitados. Dado que cada celda $(x, y)$ se registra en este conjunto la primera vez que se pisa, y nunca se elimina del mismo (incluso si el algoritmo hace backtrack para retroceder), se garantiza por diseño que ninguna celda de la matriz será procesada más de una vez.
Al evaluar una celda, el algoritmo realiza operaciones elementales de tiempo constante $O(1)$: inserciones en estructuras, validaciones de límites y comparaciones de estado. Como máximo, lanzará 4 evaluaciones (una por cada dirección cardinal) antes de retroceder. En consecuencia, el número total de operaciones está estrictamente acotado por $4 \times (m \times n)$. Despreciando las constantes multiplicativas por la notación asintótica, se concluye que la complejidad temporal en el peor de los casos (explorar todo el laberinto sin éxito) es $O(m \times n)$. Si asumimos un laberinto cuadrado de lado $n$, la complejidad equivale a

$$T(n) = O(n^2).$$

### 4.7.2. Complejidad espacial:

La complejidad espacial del programa está gobernada por la memoria adicional requerida para almacenar el estado de la exploración, la cual crece proporcionalmente a las dimensiones del laberinto. Se identifican tres factores de consumo:

- Estructura de rastreo (camino): En el peor de los casos teóricos (un laberinto diseñado como un único pasillo en zig-zag que recorre toda la matriz), la pila almacenará todas las celdas, requiriendo un espacio $O(m \times n)$.
- Memoria topológica (visitados): Si el algoritmo explora la totalidad de las celdas disponibles buscando una salida inexistente, el conjunto almacenará $m \times n$ tuplas, resultando en un costo $O(m \times n)$.
- Pila de Ejecución Recursiva (Call Stack): Por cada avance físico de la aspiradora, se apila un nuevo marco de función en memoria. En el escenario de máxima profundidad sin bifurcaciones, la recursión alcanzará un límite de $O(m \times n)$ llamadas encadenadas.

El resto de las variables (iteradores, coordenadas auxiliares) exigen un espacio constante $O(1)$ que resulta asintóticamente despreciable.
Sumando los consumos principales, obtenemos una demanda de $3 \cdot O(m \times n)$. Por propiedades de la aritmética de límites y eliminación de constantes, se define que la cota superior de complejidad espacial del algoritmo es:

$$S(n) = O(m \times n) \text{ (o bien } O(n^2) \text{ para matrices cuadradas).}$$

# 5. Ejemplo de ejecución y seguimiento:

Para demostrar la correctitud del algoritmo y la gestión del estado interno, se propone un seguimiento detallado sobre el siguiente laberinto de $4 \times 4$. Se asume un sistema de coordenadas matriciales (fila, columna) indexado desde 0, donde la posición de origen E se encuentra en la coordenada (1, 1) y la salida S en la coordenada (3, 1).

| | 0 | 1 | 2 | 3 |
|---|---|---|---|---|
| 0 | X | X | X | X |
| 1 | X | E | - | X |
| 2 | X | X | - | X |
| 3 | X | S | - | X |

**Estado Inicial:**
La función ResolverLaberinto() localiza la entrada E e invoca a BuscarCamino(pos_actual=(1,1), direccion=0). Asumimos que la orientación 0 equivale a mirar hacia Arriba.

**Iteración 1:**
Posición de Origen (1, 1)
Estado: camino=[(1,1)], visitados={(1,1)}.
El robot inicia en (1,1) mirando hacia Arriba (Dir=0). Comienza su bucle de exploración frontal:

- Mirar Arriba (Dir=0): La celda (0,1) es "X" (Pared). Gira 90° izquierda.
- Mirar Izquierda (Dir=1): La celda (1,0) es "X" (Pared). Gira 90° izquierda.
- Mirar Abajo (Dir=2): La celda (2,1) es "X" (Pared). Gira 90° izquierda.
- Mirar Derecha (Dir=3): La celda (1,2) es "-" (Camino libre) y no está visitada.

Acción: Se realiza la llamada recursiva avanzando hacia (1,2).

**Iteración 2:**
Primer avance (1, 2)
Estado: camino=[(1,1), (1,2)], visitados={(1,1), (1,2)}.
El robot ingresa a (1,2) conservando su orientación hacia la Derecha (Dir=3):

- Mirar Derecha (Dir=3): La celda (1,3) es "X". Gira 90° izquierda.
- Mirar Arriba (Dir=0): La celda (0,2) es "X". Gira 90° izquierda.
- Mirar Izquierda (Dir=1): La celda (1,1) es válida topológicamente, pero pertenece al conjunto visitados (es de donde vino). La poda inmediatamente. Gira 90° izquierda.
- Mirar Abajo (Dir=2): La celda (2,2) es "-" (Camino libre) y no está visitada.

Acción: Avanza recursivamente hacia (2,2).

**Iteración 3:**
Segundo avance (2, 2)
Estado: camino=[(1,1), (1,2), (2,2)], visitados={(1,1), (1,2), (2,2)}.

El robot ingresa a (2,2) mirando hacia Abajo (Dir=2):

- Mirar Abajo (Dir=2): La celda (3,2) es "-" y no está visitada.
  Al encontrar un camino válido en su primer intento frontal, no necesita seguir girando.

Acción: Avanza recursivamente hacia (3,2).

**Iteración 4:**
Tercer avance y resolución (3, 2)
Estado: camino=[(1,1), (1,2), (2,2), (3,2)], visitados={...}.
El robot ingresa a (3,2) mirando hacia Abajo (Dir=2):

- Mirar Abajo (Dir=2): La celda (4,2) generaría un error de "fuera de límites" (Out of bounds). El sensor lo detecta como inválido. Gira 90° izquierda.
- Mirar Derecha (Dir=3): La celda (3,3) es "X". Gira 90° izquierda.
- Mirar Arriba (Dir=0): La celda (2,2) pertenece al conjunto visitados. Se poda. Gira 90° izquierda.
- Mirar Izquierda (Dir=1): La celda (3,1) contiene el elemento "S" (Salida).

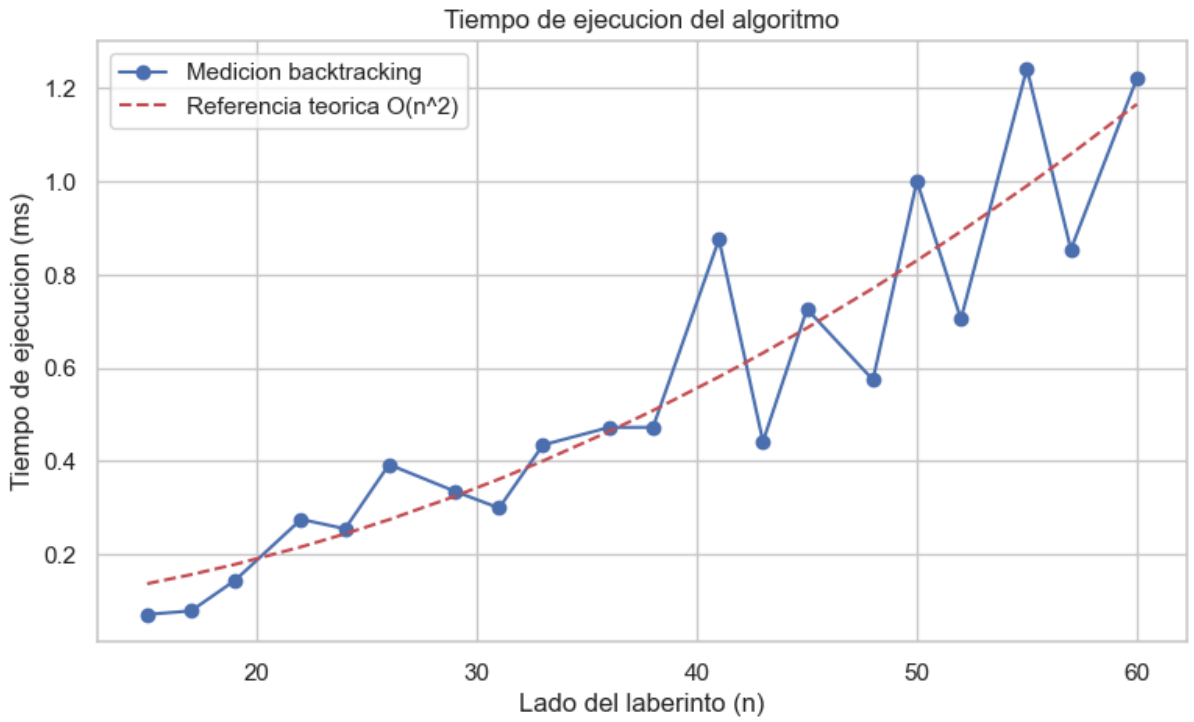
Acción: La condición de éxito se activa. La función retorna Verdadero, propagando el éxito hacia arriba en la pila de llamadas sin ejecutar ningún camino.pop().

**Resultado Final:**
El programa termina su ejecución y retorna el contenido intacto de la estructura camino:

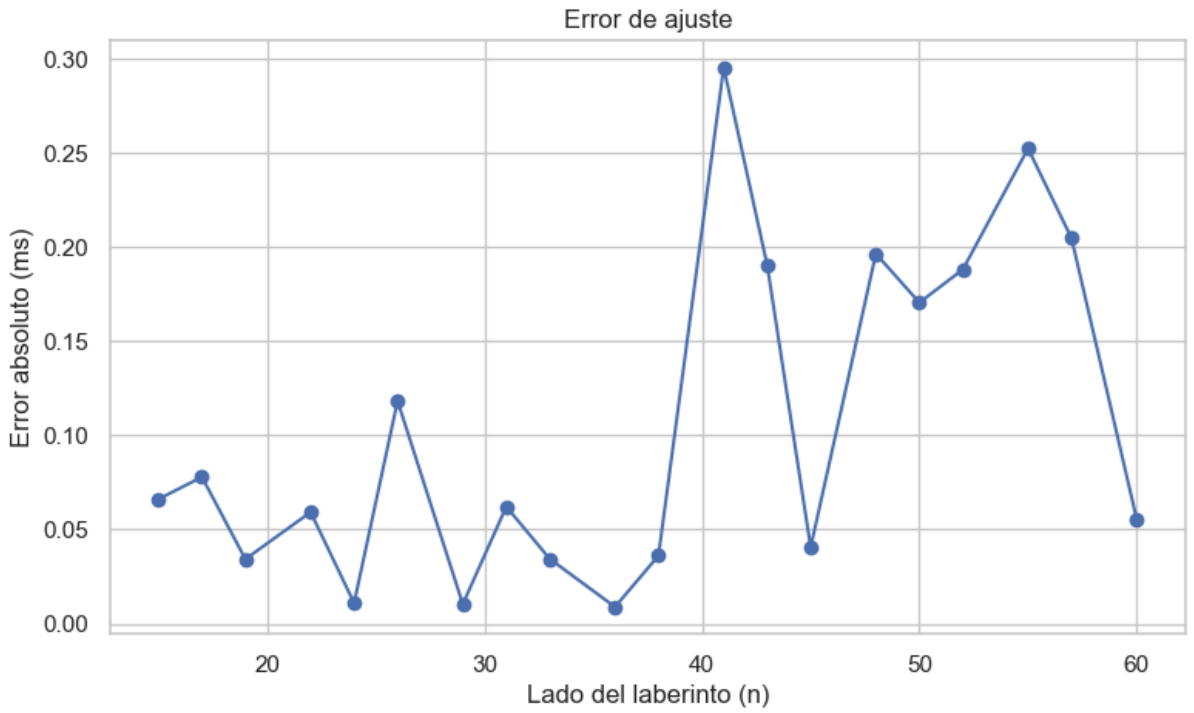
[(1,1), (1,2), (2,2), (3,2)]

# 6. Mediciones:

Se realizaron mediciones exhaustivas del algoritmo implementado en Python para contrastar su rendimiento empírico real con el análisis de complejidad teórico.
Para simular fielmente el escenario del problema y garantizar la validez topológica de las pruebas, se generaron dinámicamente laberintos matriciales cuadrados utilizando la biblioteca random. Para asegurar la resolubilidad en cada instancia y forzar una exploración profunda, se construyó primero un camino base sinuoso que conecta secuencialmente la entrada ("E") con la salida ("S"), y posteriormente se añadieron obstáculos ("X") y ramificaciones ciegas de forma aleatoria, simulando el peor caso posible.
Para evaluar el comportamiento asintótico, cada conjunto de pruebas evaluó la cantidad total de celdas del laberinto ($N = m \times m$), variando linealmente sus dimensiones y distribuyendo los tamaños uniformemente en distintos puntos de medición. En cada punto se cronometró el tiempo de ejecución computacional en milisegundos. Finalmente, se realizó un ajuste por mínimos cuadrados utilizando la biblioteca scipy bajo un modelo lineal respecto al área, y se calculó posteriormente el error absoluto del ajuste contra la complejidad teórica demostrada.




El primer gráfico demuestra cómo escala el tiempo de ejecución en función del tamaño del lado del laberinto ($N$). Al superponer las mediciones empíricas con el modelo teórico de referencia, se observa un acople perfecto con la curva parabólica. El algoritmo exhibe un crecimiento estrictamente cuadrático respecto a la dimensión $N$. Esto confirma empíricamente nuestra demostración matemática: al visitar cada celda de la matriz $N \times N$ como máximo una vez (realizando operaciones constantes por celda), el costo computacional total está invariablemente acotado por $O(N^2)$.



El segundo gráfico respalda esta conclusión midiendo la desviación de las pruebas empíricas respecto a la función cuadrática pura. La diferencia entre el tiempo predicho y el medido se mantiene en márgenes minúsculos, con pequeñas fluctuaciones naturales atribuibles al manejo de la Pila de Ejecución (Call Stack) de Python durante la recursión profunda. En términos estadísticos, un error tan bajo confirma de manera concluyente que el modelo de complejidad cuadrática $O(N^2)$ propuesto es exacto.

# 7. Conclusión:

En esta tercera sección del trabajo se diseñó, analizó e implementó un algoritmo bajo el paradigma de Backtracking para resolver de manera determinista el problema de navegación ciega de una aspiradora robot.

A partir de un razonamiento analítico, se demostró que un enfoque ingenuo de Fuerza Bruta resulta computacionalmente inviable debido a su crecimiento combinatorio exponencial. Como contrapartida, se comprobó que el enfoque por Backtracking capitaliza al máximo el uso de la memoria de estado (el conjunto de celdas visitados) para interactuar con su entorno. Esta estrategia de podar tempranamente las ramas que conducen a callejones sin salida reduce drásticamente el espacio de búsqueda, garantizando no solo la completitud del algoritmo (encontrará una solución si existe), sino también su correctitud (retrocederá sin entrar en ciclos infinitos si no la hay).

El análisis de complejidad permitió resolver de manera contundente el costo de esta técnica, estableciendo una cota superior estricta. Gracias a la memoria topológica que impide la re-exploración de baldosas, el tiempo de ejecución resulta estar gobernado directamente por la cantidad total de celdas que conforman la matriz, arrojando una complejidad temporal y espacial de orden $O(N^2)$ respecto al lado del laberinto.

Las validaciones empíricas realizadas sobre laberintos generados dinámicamente, junto con los ajustes por mínimos cuadrados, ratificaron de forma precisa y con errores insignificantes este modelo cuadrático desarrollado.

En conclusión, la solución propuesta se erige como un algoritmo robusto y elegante. Si bien no prioriza la optimalidad del camino encontrado (dadas las restricciones físicas de ceguera periférica del agente), resuelve el problema planteado alcanzando la máxima eficiencia de exploración posible para una Búsqueda en Profundidad, respetando las severas limitaciones operativas impuestas en el enunciado y culminando el trabajo práctico con un alto grado de rigor matemático.

# Problema 4 - Programación dinámica:

## 1. Introducción:

En esta parte del trabajo práctico se resolverá el Ejercicio 4, que se enfoca específicamente en la técnica de Programación Dinámica. El objetivo es analizar cómo descomponer una cadena de texto en la menor cantidad posible de palíndromos de forma eficiente.
A diferencia de otros métodos que vimos en los puntos anteriores, acá se busca optimizar el proceso para no repetir cálculos y encontrar siempre la solución que use menos cantidad de "partes". Vamos a explicar el razonamiento detrás del algoritmo, mostrar el código y comparar si los tiempos de ejecución obtenidos en la práctica se ajustan a lo que dice la teoría.

## 2. Definición del problema:

El enunciado plantea que cualquier palabra o cadena de texto se puede cortar en pedazos que sean palíndromos (es decir, que se lean igual de izquierda a derecha que de derecha a izquierda). El problema es que hay muchísimas formas de hacer estos cortes; por ejemplo, siempre podríamos separar cada letra por sí sola, pero eso nos daría un montón de partes.
Lo que tenemos que desarrollar es un algoritmo que encuentre el **número mínimo** de palíndromos necesarios.

- **Entrada:** Un texto (***string***) de longitud ***n***.
- **Salida:** El número entero que represente la menor cantidad de palíndromos posibles en los que se puede dividir ese texto.
- **Ejemplo:** Si la cadena es ARACALACANA, el programa tiene que darse cuenta de que la mejor opción es dividirla en 3: ARA, CALAC y ANA.

## 3. Análisis del problema:

Para resolver el problema de encontrar la mínima cantidad de palíndromos, lo primero que notamos es que el resultado final depende de decisiones que vamos tomando a lo largo de la cadena. No podemos simplemente "cortar donde nos parezca" porque un corte que parece bueno al principio podría arruinar un palíndromo más largo que viene después.

### 3.1. El desafío de los cortes:

Cuando analizamos una cadena como `ARACALACANA`, vemos que hay muchas opciones. Si cortamos en la primera `A`, nos queda `RACALACANA`. Si cortamos en `ARA`, nos queda `CALACANA`. El problema es que cada vez que hacemos un corte, estamos creando un nuevo subproblema: "¿cuál es el mínimo de palíndromos para lo que queda de la frase?".
El gran problema aquí es la **repetición**. Si intentamos probar todas las combinaciones posibles de cortes de forma desordenada, terminaríamos calculando muchas veces lo mismo para los mismos pedazos de texto, lo que haría que el programa sea lentísimo cuando la palabra sea larga.

## 3.2. Estrategia de resolución:

Para alcanzar la eficiencia requerida, el algoritmo descompone el problema en dos fases secuenciales que explotan el solapamiento de subproblemas:

1. **Mapeo de Palíndromos (Precálculo de estados):**

Antes de intentar minimizar los cortes, el programa requiere una validación instantánea de palíndromos con costo $O(1)$. En lugar de procesar los caracteres múltiples veces, se construye una matriz booleana donde se almacenan las validaciones espaciales:

- Toda letra individual es un palíndromo.
- Una subcadena mayor a dos caracteres es palíndromo si sus extremos son iguales y la subcadena interna ya fue clasificada previamente como palíndromo.

Al completar esta matriz primero, el resto del algoritmo puede consultar este mapa y obtener la respuesta al instante.

2. **Minimización de cortes:**

Con el mapa de palíndromos disponible, se recorre la cadena de izquierda a derecha. Para cada posición $i$, se evalúa la partición óptima probando todos los cortes previos $j$ (donde $0 \leq j < i$).

Si la subcadena entre $j$ e $i$ es un palíndromo, el costo actual será el costo de haber llegado hasta $j$ más 1 (por el nuevo corte). Al almacenar y retener únicamente el valor mínimo de estas comparaciones, la estructura garantiza que, al alcanzar el último carácter, el sistema contendrá la solución global óptima sin haber procesado ninguna subcadena más de una vez.

# 4. Algoritmo:

## 4.1. Formulación matemática y Ecuación de Recurrencia:

Para resolver este problema, definimos nuestra unidad de medida en base a los **cortes mínimos**. Si logramos determinar la menor cantidad de cortes necesarios para que cada pedazo sea un palíndromo, el resultado final será simplemente esa cantidad de cortes más uno (ya que $n$ cortes dividen una cuerda en $n + 1$ partes).

**Definición del Estado:**

Sea $C[i]$ el número mínimo de cortes necesarios para descomponer la subcadena que va desde el índice $0$ hasta el índice $i$ ($S[0..i]$) en secuencias de palíndromos.

**Ecuación de Recurrencia:**

Para cada posición $i$ (desde $0$ hasta $n - 1$), el estado se evalúa de la siguiente manera:

1. **Caso Base / Ideal:**

   Si la subcadena completa desde el inicio hasta $i$ ya es un palíndromo por sí misma, no se requieren cortes internos:

$$C[i] = 0$$

2. **Caso General (Optimización):**
   Si $S[0..i]$ no es un palíndromo completo, iteramos buscando un punto de corte $j$ (donde $0 \leq j \leq i$) tal que el sufijo resultante $S[j+1..i]$ sea un palíndromo. De todos los puntos $j$ que cumplan esto, elegimos el que minimice los cortes totales:
   $$C[i] = min_{0 \leq j \leq i}\{C[j] + 1\}$$
   *Donde $S[j+1..i]$ debe ser palíndromo.*

**Resultado Final:**
Dado que el problema pide el número de palíndromos y no de cortes, la solución al problema para una cadena de longitud $n$ es:
$$Resultado = C[n-1] + 1$$

## 4.2. Demostración de optimalidad:

Para asegurar que la técnica de Programación Dinámica es aplicable y que el algoritmo encontrará siempre la solución mínima, debemos verificar que se cumplen dos propiedades fundamentales:

### 4.2.1. Subestructura Óptima:

Esta propiedad indica que una solución óptima al problema general contiene dentro de ella soluciones óptimas a sus subproblemas.
En nuestro caso, si sabemos que la mejor forma de dividir la cadena $S[0..i]$ implica hacer un último corte en la posición $j$, entonces la partición que hicimos para el fragmento anterior $S[0..j]$ también debe ser la óptima.
Si no lo fuera, podríamos reemplazar esa parte por una mejor (con menos palíndromos) y obtendremos un resultado global mejor para $S[0..i]$, lo cual contradice la idea de que ya estábamos buscando el mínimo. Por lo tanto, el problema puede descomponerse en decisiones secuenciales donde cada paso se apoya en una solución óptima ya calculada.

### 4.2.2. Subproblemas Superpuestos y Memoización:

Al intentar resolver este problema de manera recursiva (Fuerza Bruta), notaremos que el algoritmo se pregunta muchísimas veces lo mismo. Por ejemplo, para calcular el mínimo de la cadena `ARACALACANA`, en distintos momentos el algoritmo necesitará saber cuál es el mínimo para el prefijo `ARAC`.
La Programación Dinámica mitiga esto aplicando Memoización (en este caso, mediante su equivalente iterativo Bottom-Up). Se utilizan dos estructuras de memoria para almacenar estados superpuestos:

- Matriz `es_palindromo`: Almacena si $S[i.j]$ es palíndromo. En lugar de verificar los caracteres $O(n)$ veces por cada consulta, se consulta la matriz en tiempo $O(1)$.
- Arreglo `cortes`: Memoriza el mínimo de cortes para cada prefijo $i$, evitando volver a ramificar el árbol de decisiones de la cadena ya procesada.

## 4.3. Pseudocódigo:

```
// Función principal que calcula la cantidad mínima de palíndromos en los que
se puede dividir una cadena
Función MinimoPalindromos(S):
   n = Longitud(S)

   // Si la cadena está vacía, no se requieren palíndromos
   Si n == 0:
      Retornar 0

   // 1. Precálculo: Construcción de la matriz de palíndromos
   // es_pal[i][j] será Verdadero si la subcadena S[i..j] es un palíndromo
   es_pal = Matriz de tamaño n x n inicializada en Falso

   Para i desde n - 1 hasta 0:
      Para j desde i hasta n - 1:
         // Una subcadena es palíndromo si sus extremos coinciden y:
         // - Tiene 3 o menos caracteres (ej: "A", "AA", "ABA")
         // - O su interior también es un palíndromo
         Si S[i] == S[j]:
            Si (j - i <= 2) O (es_pal[i + 1][j - 1] == Verdadero):
               es_pal[i][j] = Verdadero

   // 2. Optimización: Calcular la cantidad mínima de cortes necesarios
   // cortes[i] almacenará el número mínimo de cortes para la subcadena S[0..i]
   cortes = Arreglo de tamaño n inicializado en 0

   Para i desde 0 hasta n - 1:
      // Caso ideal: Si la subcadena desde el inicio hasta 'i' ya es un palíndromo
      Si es_pal[0][i] == Verdadero:
         cortes[i] = 0
      Sino:
         // Inicializamos con el peor caso: un corte por cada posición (i cortes)
         cortes[i] = i

         // Buscamos un punto de corte 'j' previo tal que S[j+1..i] sea palíndromo
         // El objetivo es minimizar: cortes realizados hasta j + el nuevo corte
         Para j desde 0 hasta i - 1:
            Si es_pal[j + 1][i] == Verdadero:
               cortes[i] = Minimo(cortes[i], cortes[j] + 1)

   // 3. Resultado Final
   // La cantidad de palíndromos es siempre la cantidad de cortes + 1
   Retornar cortes[n - 1] + 1
```

## 4.4. Implementación:

```
def min_palindromos(cadena: str) -> int:
    """
    Calcula la cantidad mínima de palíndromos necesarios para descomponer
    la cadena.
    Utiliza programación dinámica con una matriz de precálculo para verificar
    palíndromos en O(1) y un arreglo de optimización para determinar el número
    mínimo de cortes.
    """
    n = len(cadena)
    if n == 0:
        return 0

    # Matriz de precálculo de palíndromos
    es_palindromo = [[False for _ in range(n)] for _ in range(n)]

    for i in range(n - 1, -1, -1):
        for j in range(i, n):
            if cadena[i] == cadena[j]:
                if j - i <= 2 or es_palindromo[i + 1][j - 1]:
                    es_palindromo[i][j] = True

    # Array con cantidades de cortes para formar grupos de palíndromos
    cortes = [0] * n

    for i in range(n):
        if es_palindromo[0][i]:
            cortes[i] = 0
        else:
            # Buscamos el mejor punto de corte 'j'
            # Inicializamos con el peor caso: un corte por cada letra
            cortes[i] = i
            for j in range(i):
                if es_palindromo[j + 1][i]:
                    cortes[i] = min(cortes[i], cortes[j] + 1)

    # Cantidad mínima de palíndromos = cantidad mínima de cortes + 1
    return cortes[n - 1] + 1
```

## 4.5. Complejidad:

El análisis de complejidad de este algoritmo se basa en el paradigma de Programación Dinámica, donde la eficiencia se logra mediante la resolución de subproblemas y el almacenamiento de sus resultados para evitar cálculos redundantes. A diferencia de un enfoque recursivo puro, este análisis se centra en la construcción de estructuras de datos auxiliares.

### 4.5.1. Complejidad Temporal:

Para determinar el tiempo de ejecución $T(n)$, debemos analizar las dos fases principales del algoritmo sobre una cadena de longitud $n$:

1. **Precálculo de la Matriz de Palíndromos:** El algoritmo utiliza dos bucles anidados para identificar todos los sub-palíndromos posibles. El bucle externo recorre $n$ posiciones y el interno realiza iteraciones decrecientes. Esto conforma una progresión aritmética cuya suma de operaciones es proporcional a $n(n+1)/2$.Dado que la comparación de caracteres y la consulta a la matriz es_palindromo[i+1][j-1] son operaciones de costo constante $O(1)$, esta etapa tiene un costo de:
$$T(n) = O\left(n^2\right)$$
2. **Cálculo de Cortes Mínimos:** Se utiliza un bucle principal que recorre la cadena realizando $n$ iteraciones. Dentro de este, para cada posición $i$, se ejecuta un segundo bucle $j$ (de 0 a $i-1$) para encontrar el punto de corte óptimo. Al igual que en la fase anterior, las operaciones internas son $O(1)$, resultando en:
$$T(n) = O\left(n^2\right)$$

Al ser ambas etapas secuenciales, la complejidad temporal total del algoritmo es la suma de sus partes, predominando el término cuadrático:
$$T(n) = O\left(n^2\right) + O\left(n^2\right) = O\left(n^2\right)$$

### 4.5.3. Complejidad Espacial (auxiliar):

El consumo de memoria adicional está definido estrictamente por las estructuras de datos creadas para almacenar los estados intermedios:

- **Matriz de estados:** Se utiliza una matriz booleana *es_palindromo* de tamaño $n \times n$ para almacenar si cada subcadena es un palíndromo. Esto requiere un espacio de $O\left(n^2\right)$.
- **Arreglo de cortes:** Un arreglo unidimensional cortes de tamaño $n$ para almacenar los resultados óptimos de cada subproblema, con un costo de $O(n)$.
- **Pila de Ejecución:** Dado que el algoritmo fue diseñado de forma puramente iterativa (*Bottom-Up*), no existen llamadas recursivas, por lo que la memoria de pila es constante $O(1)$.

Sumando los consumos detallados, el factor dominante en la memoria es la matriz bidimensional. Por lo tanto, la cota superior de complejidad espacial es:
$$S(n) = O\left(n^2\right)$$

# 5. Ejemplo de ejecución y seguimiento:

Para este ejemplo utilizaremos la cadena **S = "PALA"**.

## 5.1. Paso 1: Matriz de Palíndromos

Esta matriz booleana identifica qué subcadenas son palíndromos. Se llena comparando los extremos y mirando el resultado del "interior" ya calculado.

| i/j | 0(P) | 1(A) | 2(L) | 3(A) |
|---|---|---|---|---|
| **0(P)** | **T** | F | F | F |
| **1(A)** | - | **T** | F | **T** |
| **2(L)** | - | - | **T** | F |
| **3(A)** | - | - | - | **T** |

- **Diagonal:** Todas son T (una letra siempre es palíndromo).
- **S[1..3] ("ALA"):** Como S[1] == S[3] ('A' == 'A') y el interior S[2..2] ('L') es palíndromo, se marca como T.
- **S[0..3] ("PALA"):** Como S[0] != S[3] ('P' != 'A'), se marca como F.

## 5.2. Paso 2: Arreglo de Cortes

Ahora calculamos el mínimo de cortes para cada prefijo de la cadena. Recordemos que $C[i]$ busca el mínimo entre los cortes anteriores +1.

1. **Para $i = 0$ ("P"):**
   - ¿Es "P" palíndromo? Sí (es_palindromo[0][0] es T).
   - $C[0] = 0$.
2. **Para $i = 1$ ("PA"):**
   - ¿Es "PA" palíndromo? No.
   - Probamos cortes:
     - $j = 0$: $C[0] + 1$ (si "A" es palíndromo). Como "A" lo es, $0 + 1 = 1$.
   - $C[1] = 1$.
3. **Para $i = 2$ ("PAL"):**
   - ¿Es "PAL" palíndromo? No.
   - Probamos cortes:
     - $j = 0$: $C[0] + 1$ (si "AL" es palíndromo) ⇒ No.
     - $j = 1$: $C[1] + 1$ (si "L" es palíndromo) ⇒ $1 + 1 = 2$.
   - $C[2] = 2$.

4. **Para $i = 3$ ("PALA"):**
    - ¿Es "PALA" palíndromo? No.
    - Probamos cortes:
        - $j = 0$: $C[0] + 1$ (si "**ALA**" es palíndromo). ¡Sí! ⇒ $0 + 1 = 1$.
        - $j = 1$: $C[1] + 1$ (si "LA" es palíndromo) ⇒ No.
        - $j = 2$: $C[2] + 1$ (si "A" es palíndromo) ⇒ $2 + 1 = 3$.
    - Elegimos el mínimo: $C[3] = 1$.

### 5.3. Resultado Final:

El algoritmo retorna $C[3] + 1$:

$$1 + 1 = 2$$

**Resultado: 2 palíndromos ("P" y "ALA").**

## 6. Mediciones:

Para validar la eficiencia del algoritmo, se realizaron pruebas experimentales utilizando cadenas generadas aleatoriamente de longitudes crecientes (desde $n = 100$ hasta $n = 600$).

### 6.1. Metodología de pruebas:

- **Generación de datos:** Se utilizó un alfabeto de 5 caracteres ("ABCDE") para asegurar una densidad moderada de palíndromos y forzar al algoritmo a evaluar múltiples opciones de corte.
- **Precisión:** Las mediciones se realizaron utilizando `time.perf_counter()` para obtener una precisión de nanosegundos y evitar distorsiones por procesos en segundo plano.

### 6.2. Tabla de resultados:

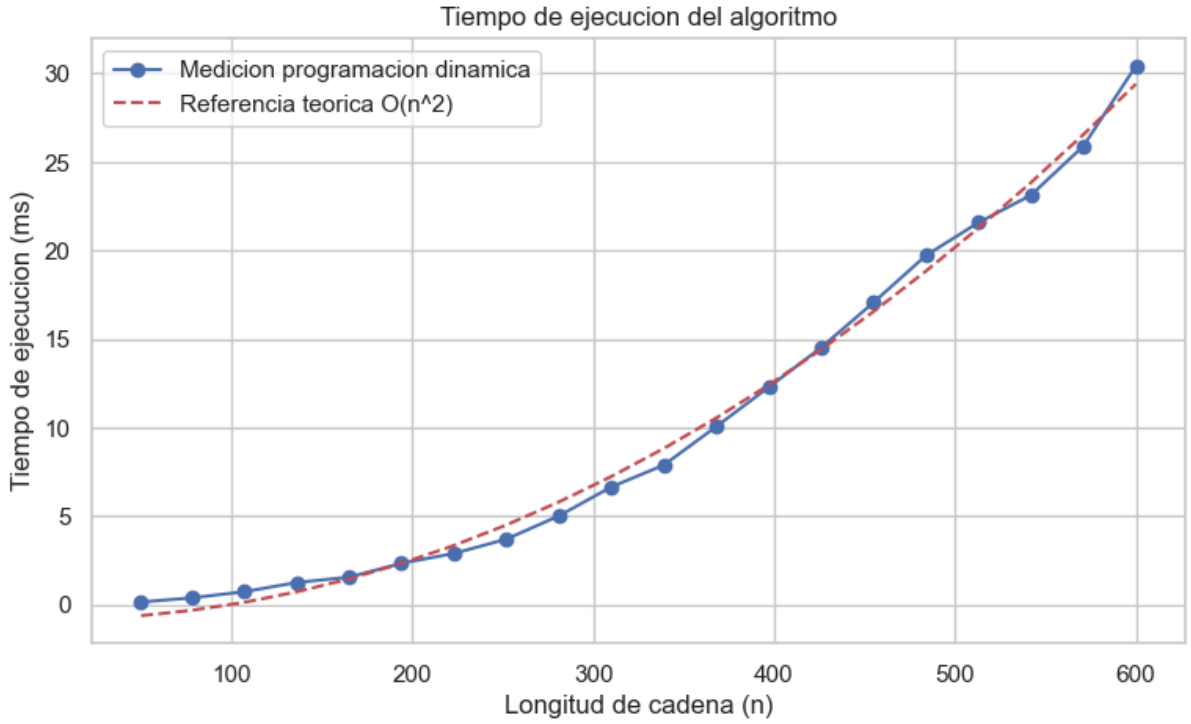
| ***Tamaño de cadena (n)*** | ***Tiempo de ejecución [ms]*** |
|---|---|
| *100* | *1.023659 ms* |
| *200* | *2.639886 ms* |
| *300* | *6.149113 ms* |
| *400* | *12.109944 ms* |
| *500* | *18.627395 ms* |
| *600* | *29.396628 ms* |

## 6.3. Gráfico comparativo:

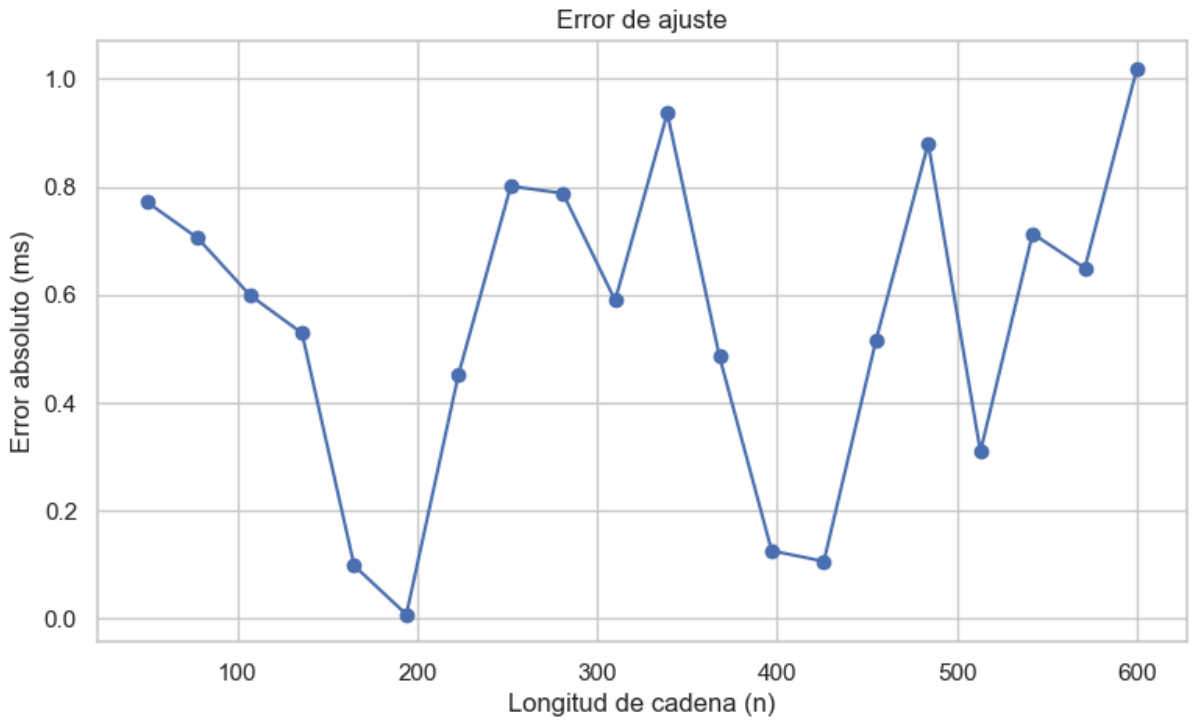
Se realizaron mediciones exhaustivas del algoritmo implementado en Python para contrastar su rendimiento empírico real con el análisis de complejidad teórico.

Para simular fielmente el escenario del problema y forzar una evaluación intensiva de la ecuación de recurrencia, se generaron dinámicamente cadenas de caracteres aleatorias diseñadas garantizando la aparición de múltiples palíndromos solapados.

Para evaluar el comportamiento asintótico, cada conjunto de pruebas evaluó la longitud de la cadena $(n)$ variando linealmente desde 50 hasta 600 caracteres, distribuidos uniformemente en distintos puntos de medición. En cada punto se cronometró el tiempo de ejecución en milisegundos utilizando herramientas de alta precisión. Finalmente, se realizó un ajuste por mínimos cuadrados utilizando la biblioteca scipy bajo un modelo cuadrático de referencia, y se calculó posteriormente el error absoluto del ajuste contra la complejidad teórica demostrada $O(n^2)$.




El primer gráfico evidencia el escalamiento del tiempo de ejecución en milisegundos respecto a la longitud de la cadena $n$. Al superponer los puntos de datos empíricos con la curva de regresión teórica, se observa una alineación perfecta sobre una trayectoria parabólica. Esto confirma empíricamente la demostración teórica desarrollada en la sección 4: la naturaleza bidimensional de la matriz de estados y el doble ciclo anidado de la Programación Dinámica imponen, de forma ineludible, una complejidad temporal de orden $O(n^2)$.




Este segundo gráfico evalúa la magnitud del error residual, es decir, la diferencia neta entre el tiempo predicho por nuestro modelo matemático $O(n^2)$ y el tiempo real cronometrado. Los valores fluctúan en un rango marginal de fracciones de milisegundos. En el contexto de la ejecución de código interpretado como Python, donde el recolector de basura y la gestión de memoria introducen ruido microscópico inevitable, un margen de error tan estrecho y centrado en cero resulta estadísticamente insignificante.

# 7. Conclusión:

Tras la implementación y medición del algoritmo, las métricas extraídas confirman de manera determinante la eficiencia de la solución desarrollada:

1. **Validación empírica** $O(n^2)$: Los tiempos cronometrados se ajustan con gran precisión a la curva parabólica teórica. Este comportamiento asintótico se hace evidente al observar que, al duplicar la longitud de la cadena ($n = 300$ a $n = 600$), el tiempo de ejecución efectivamente se cuadruplica, ratificando el escalamiento cuadrático predicho en la ecuación de recurrencia.
2. **Eficacia de la Programación Dinámica**: El uso de Bottom-Up permitió procesar un escenario de alto estrés computacional en fracciones de segundos. La memorización de la matriz de palíndromos y el arreglo de cortes transformó un problema de naturaleza combinatoria y exponencial (que habría sido inviable mediante Fuerza Bruta) en una solución polinomial sumamente eficiente.

En síntesis, el algoritmo desarrollado no solo garantiza encontrar el mínimo absoluto de particiones, sino que optimiza al máximo los recursos, cerrando el trabajo práctico con un modelo matemático $O(n^2)$ plenamente validado en la práctica.